

# Deep learning

## Part I

Olivier Caelen

# Table of contents

- Single-layer Neural Networks
  - Linear regression
  - Learning & gradient descent
  - Perceptron
- Multilayer perceptron
  - Architecture
  - Learning & backpropagation
  - Callbacks
  - Regularization
- Convolution neural network
  - Convolutional layers
  - Pooling layers
  - Some common deep learning architectures
- Deep generative modeling
  - Autoencoder
  - Variational autoencoder
  - Generative Adversarial Networks (GAN)
- Natural Language Processing (NLP)
  - Tokenization
  - Normalization
  - Bag of Words & TF-IDF
- Embedding & word2vec (Text processing)
- Recurrent Neural Networks (Deep sequence modeling)
  - Simple Recurrent Networks (SRN)
  - Long short-term memory (LSTM)
- Attention Mechanism and Transformer Networks
- Large language models
- Graph neural networks (GNN)

# What is deep learning

## Artificial Intelligence

Any technique that allows the computer to imitate human behavior



## Machine Learning

Ability to learn without being explicitly programmed



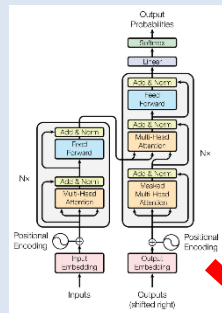
## Deep learning

Extract patterns from data using artificial neural networks



## Transformers

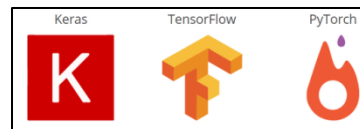
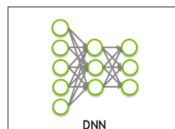
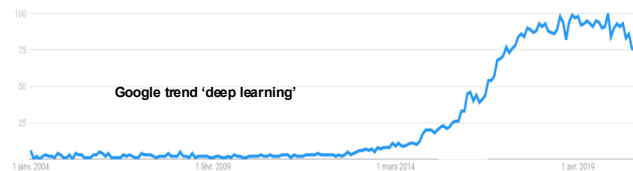
Models for sequence-to-sequence tasks using attention mechanisms



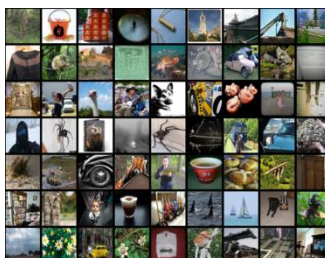
GPT is based on Transformers

# Deep learning Neural networks

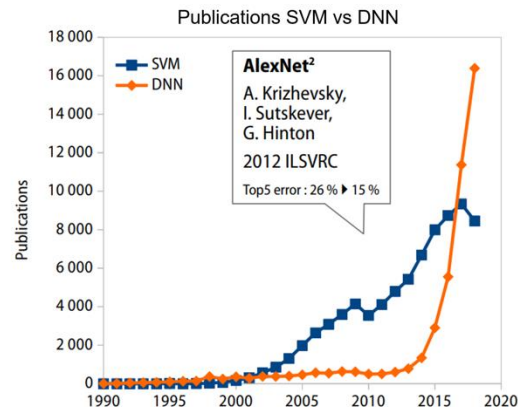
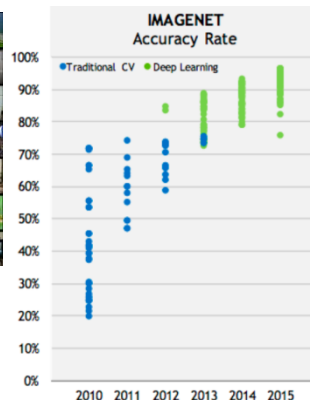
- Deep learning systems are neural networks
  - Exist since the 70s (and even before).
- What has changed:
  - more data
  - new hardware
  - new algorithms
  - new software

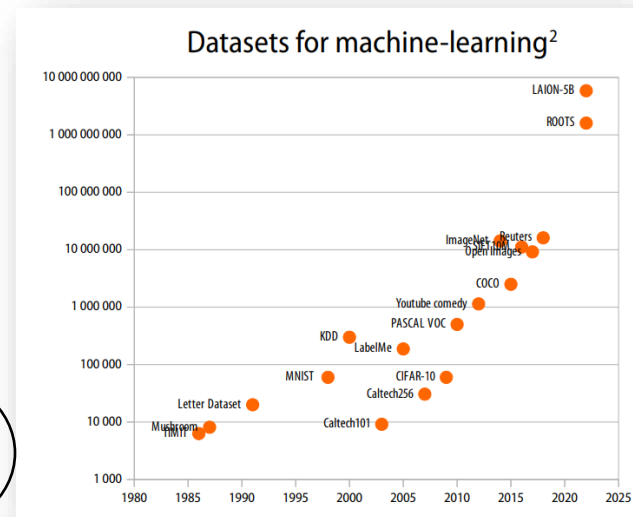
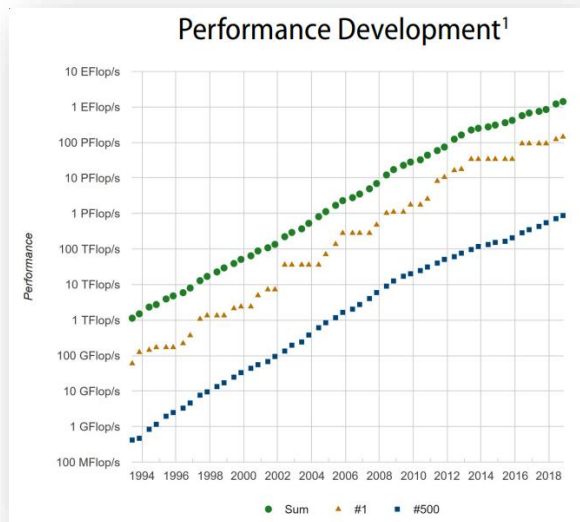


14,197,122 indexed images



IMAGENET





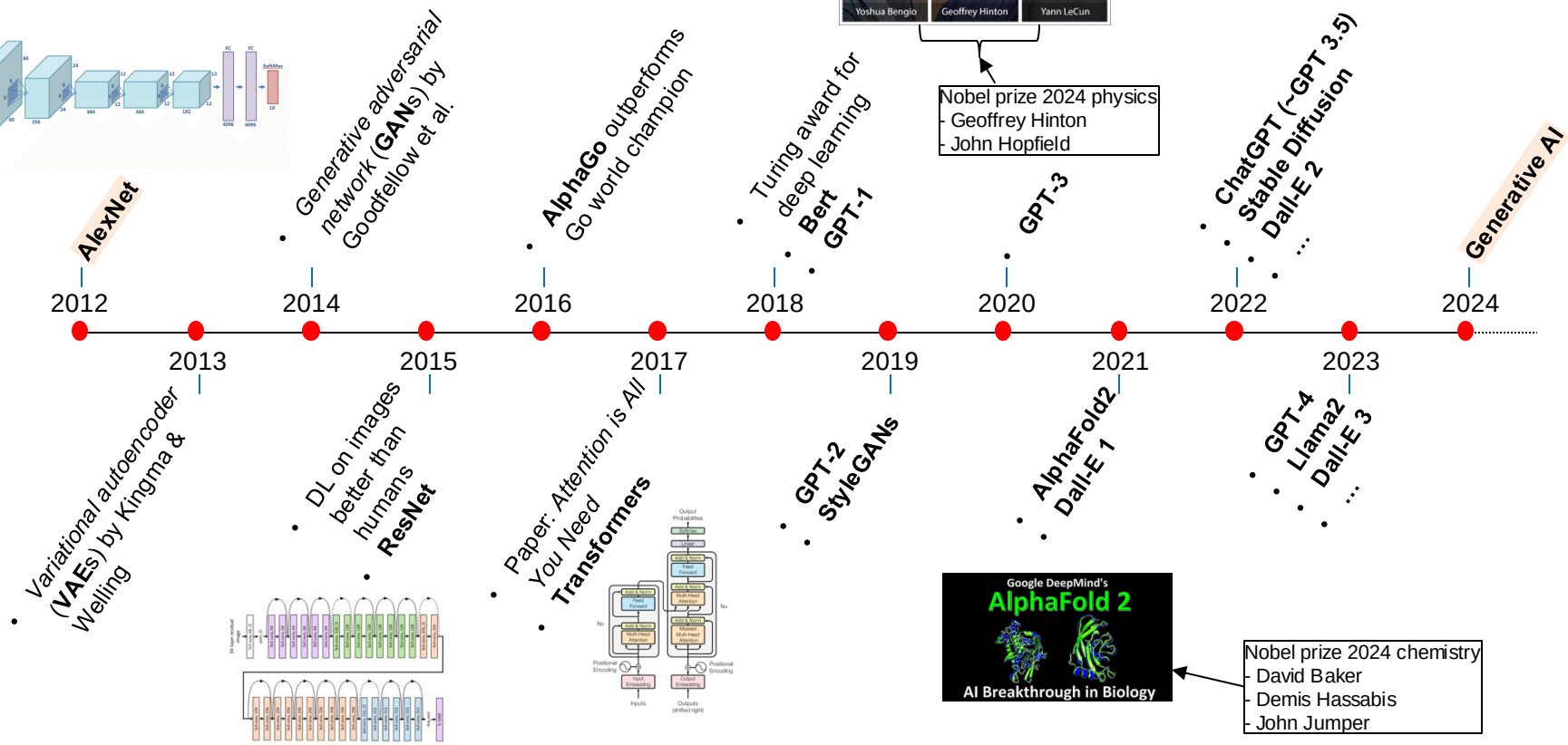
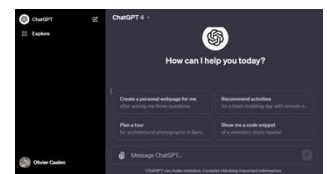
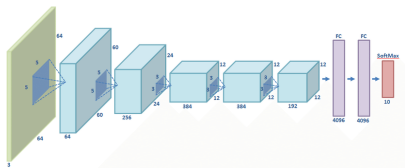
Source: TOP500 List - <https://www.top500.org/>

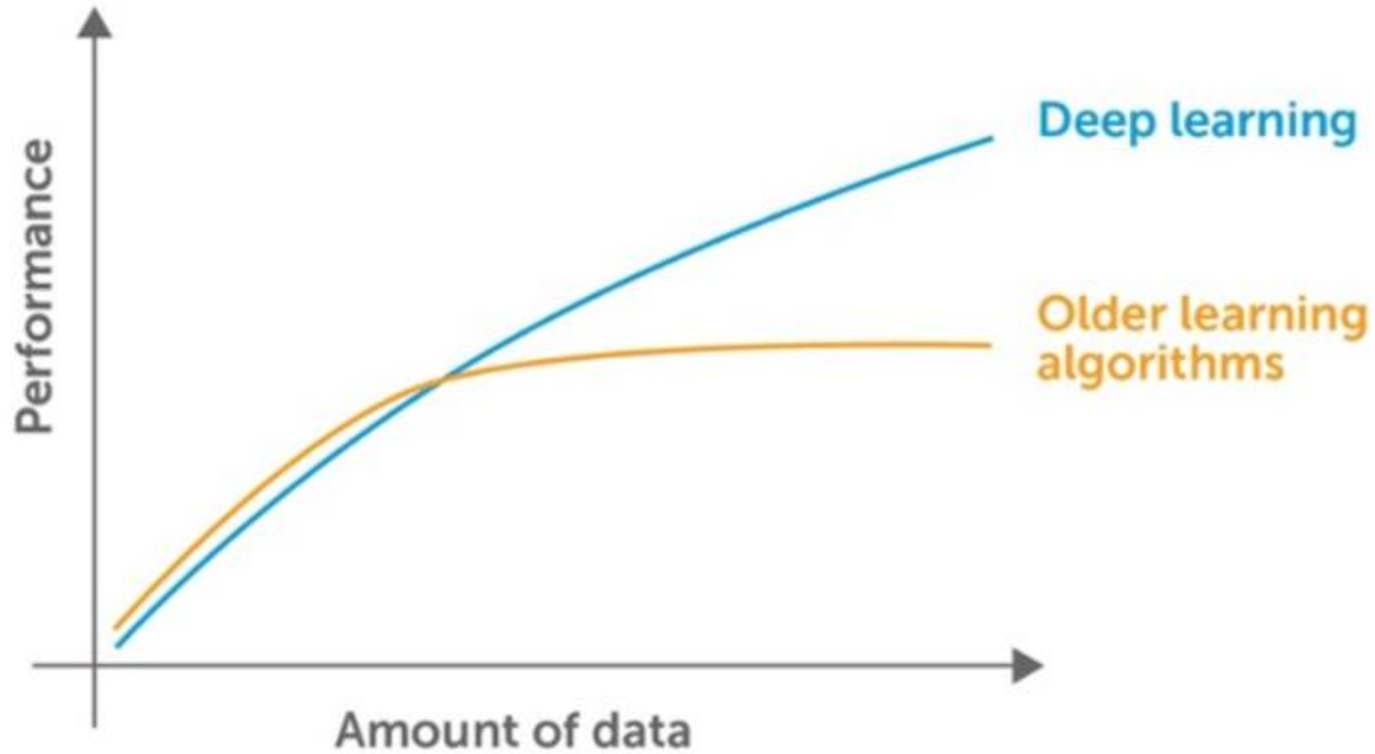
Source: Wikipedia

In 25  
years

$\times 10^6$   
Flops

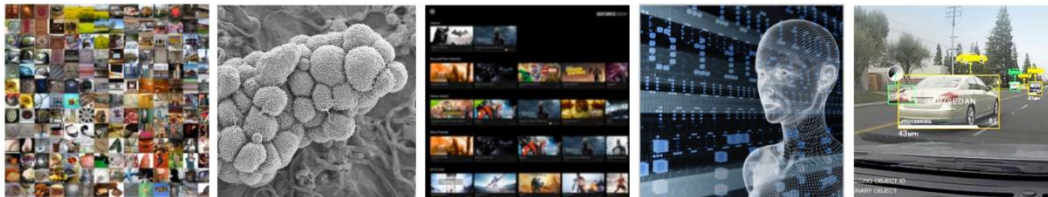
$\times 10^6$   
size





Credit: <https://medium.datadriveninvestor.com/when-not-to-use-neural-networks-89fb50622429>

# Deep learning everywhere



## INTERNET & CLOUD

Image Classification  
Speech Recognition  
Language Translation  
Sentiment Analysis  
Recommendation

## MEDICINE & BIOLOGY

Cancer Cell Detection  
Diabetic Grading  
Drug Discovery

## MEDIA & ENTERTAINMENT

Video Captioning  
Video Search  
Real Time Translation

## SECURITY & DEFENSE

Face Detection  
Video Surveillance  
Satellite Imagery

## AUTONOMOUS MACHINES

Pedestrian Detection  
Lane Tracking  
Recognize Traffic Sign

*Credit: Nvidia*

**Kaggle**

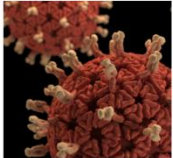
Home  
Compete  
Data  
Notebooks  
Communities  
Courses  
More

Search

### Help us better understand COVID-19

Kaggle's platform is currently hosting a variety of challenges focused on better understanding COVID-19. Join one of the below challenges to help the global community and health organizations stay informed and make data driven decisions.

View the contributions the community has already made [on this page](#).



#### Use NLP to answer key questions from the scientific literature

Answer 9 key questions using natural language processing to help the world understand COVID-19 faster. This challenge includes over 200,000 scholarly articles about COVID-19 and related coronaviruses. It was pulled together by the White House Office of Science and Technology Policy and several other partner institutions.

[Get Started](#)

AI2 @CSET

Featured Code Competition

## Deepfake Detection Challenge

Identify videos with facial or voice manipulations

[Go to Challenge](#)

Deepfake Detection Challenge · 2,265 teams · 8 months ago

[Overview](#) [Data](#) [Notebooks](#) [Discussion](#) [Leaderboard](#) [Rules](#)

**\$1,000,000**  
Prize Money

**JOTS** Technology Science

Journal Articles Type a search term Search

Advanced Search

Published on December 17, 2019 Views: 27582 Downloads: 354 Suggestions: 0

## Deepfake Bot Submissions to Federal Public Comment Websites Cannot Be Distinguished from Human Submissions

Max Weiss

Interact with Data

**Abstract**

**Introduction**

**Background**

**Methods**

**Results**

**Discussion**

**INCORRECT**

Was the following comment created by a bot or a human?

I am writing to you with regard to Idaho's proposed Medicaid waiver which has problems as it is currently written. Many Idahoans depend on Medicaid when they are sick and need help. Implementing the proposed waiver would mean taking away health care when people are most vulnerable. If someone has low income and becomes ill and cannot work that is not the time to take away their coverage.

**0%**

**8%**

**92%**

**Bot**

**Human**

Example Deepfake text generated by the bot that all survey respondents thought was from a human.

- Publicly available artificial intelligence methods can generate an enormous volume of original, human speech-like topical text "Deepfake Text" that is not based on conventional search-and-replace patterns
- I created a computer program (a bot) that generated and submitted 1,001 deepfake comments regarding a Medicaid reform waiver to a federal public comment website, stopping submission when these comments comprised more than half of all submitted comments. I then formally withdrew the bot

Cornell University

We gratefully acknowledge support from the Simons Foundation and member institutions.

arXiv.org > eess > arXiv:2004.10507

Search: All fields Search

Help | Advanced Search

### Electrical Engineering and Systems Science > Image and Video Processing

**COVID-19 e-print**

**Important:** e-prints posted on arXiv are not peer-reviewed by arXiv; they should not be relied upon without context to guide clinical practice or health-related behavior and should not be reported in news media as established information without consulting multiple experts in the field.

[Submitted on 22 Apr 2020 (v1), last revised 21 Aug 2020 (this version, v4)]

## Deep Learning for Screening COVID-19 using Chest X-Ray Images

Sanhita Basu, Sushmita Mitra, Niranjan Saha

With the ever increasing demand for screening millions of prospective "novel coronavirus" or COVID-19 cases, and due to the emergence of high false negatives in the commonly used PCR tests, the necessity for probing an alternative simple screening mechanism of COVID-19 using radiological images (like chest X-Rays) assumes importance. In this scenario, machine learning (ML) and deep learning (DL) offer fast, automated, effective strategies to detect abnormalities and extract key features of the altered lung parenchyma, which may be related to specific signatures of the COVID-19 virus. However, the available COVID-19 datasets are inadequate to train deep neural networks. Therefore, we propose a new concept called domain extension transfer learning (DETL). We employ DETL, with pre-trained deep convolutional neural network, on a related large chest X-Ray dataset that is tuned for classifying between four classes 'normal', 'normal pneumonia', 'other disease', and 'Covid - 19'. A 5-fold cross validation is performed to estimate the feasibility of using chest X-Rays to diagnose COVID-19. The initial results show promise, with the possibility of replication on bigger and more diverse data sets. The overall accuracy was measured as  $90.13\% \pm 0.14$ . In order to get an idea about the COVID-19 detection transparency, we used Grad-CAM for detecting the regions where the model paid more attention during clinical findings, as validated by experts.

**Download:**

- PDF
- Other formats

Current browse context: eess.IV

< prev | next >  
new | recent | 2004

Change to browse by: eess

References & Citations

- NASA ADS
- Google Scholar
- Semantic Scholar

Export BibTeX Citation

Bookmark



# Hello Tensorflow

[https://github.com/ocaelen/intro\\_deeplearning/blob/main/hello\\_tensorflow.ipynb](https://github.com/ocaelen/intro_deeplearning/blob/main/hello_tensorflow.ipynb)

In [1]: *# Train a simple neural network for image classification*

```
import tensorflow as tf
from tensorflow import keras
import numpy as np

print("Hello I'm tensorflow {}".format(tf.__version__))

print('Loading data...')
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train= x_train.reshape(-1,28*28)
x_train = x_train/255.

print('Make the model...')
model = keras.models.Sequential([
    keras.layers.Dense(16, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

print('Training model...')
model.fit(x_train, y_train, epochs=3, batch_size=32)

print('Model trained successfully!')
```

} Prepare the training set

} Make the model

```
Hello I'm tensorflow 2.2.0.
Loading data...
Make the model
Training model...
Epoch 1/3
1875/1875 [=====] - 14s 7ms/step - loss: 0.4853 - accuracy: 0.8591
Epoch 2/3
1875/1875 [=====] - 13s 7ms/step - loss: 0.2778 - accuracy: 0.9199
Epoch 3/3
1875/1875 [=====] - 13s 7ms/step - loss: 0.2398 - accuracy: 0.9301
Model trained successfully!
```

# Hello PyTorch

```
import torch
import torch.nn as nn
import numpy as np
from sklearn import datasets
from pickletools import optimize
import matplotlib.pyplot as plt
```

```
## Prepare data
X_numpy, y_numpy = datasets.make_regression(n_samples=200, n_features=1, noise=10, random_state=42)
X = torch.from_numpy(X_numpy.astype(np.float32))
y = torch.from_numpy(y_numpy.astype(np.float32))
y = y.view(y.shape[0], 1)

n_samples, n_features = X.shape
```

```
m = nn.Linear(n_features , 1) # input_size=1, output_size=1

loss_fct = nn.MSELoss()
optimizer = torch.optim.SGD(m.parameters(), lr=0.01)
```

```
for epoch in range(200):
    y_pred = m(X) # forward pass
    loss = loss_fct(y_pred, y)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()

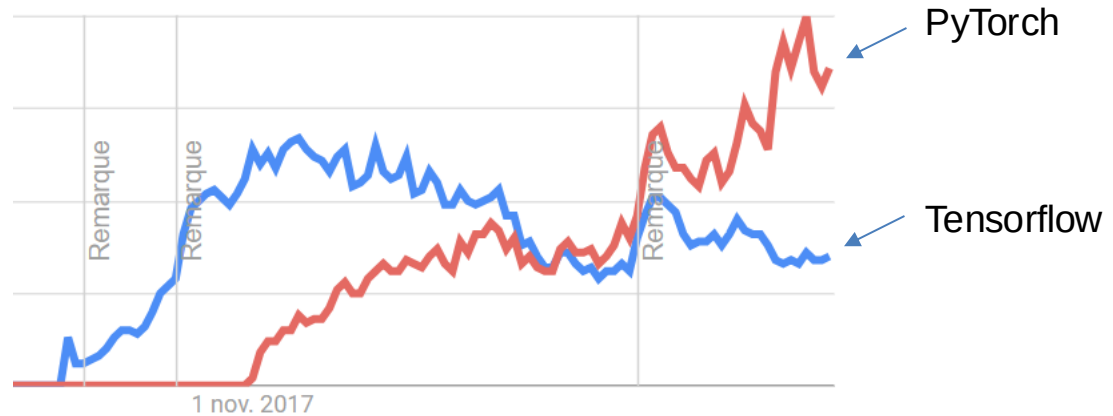
    if (epoch+1) % 40 == 0:
        print(f'epoch: {epoch+1}, loss:{loss.item():.4f}')
```

```
epoch: 40, loss:1757.2028
epoch: 80, loss:514.2151
epoch: 120, loss:204.7480
epoch: 160, loss:127.4618
epoch: 200, loss:108.1133
```

# API docs

[https://www.tensorflow.org/api\\_docs/python](https://www.tensorflow.org/api_docs/python)

<https://pytorch.org/docs/stable/>



Credit: Google Trends (Feb 2024)

## **Single-layer Neural Networks**

**(Linear regression, Perceptrons & Co...)**

# Supervised training set

Input training set

$$\mathbf{X} = \begin{pmatrix} x_1^1 & \dots & x_n^1 \\ \vdots & \ddots & \vdots \\ x_1^N & \dots & x_n^N \end{pmatrix} = \begin{pmatrix} (\mathbf{x}^1)^T \\ \dots \\ (\mathbf{x}^N)^T \end{pmatrix} = (\mathbf{x}_1 \quad \dots \quad \mathbf{x}_n) \in \mathbb{R}^{N \times n}$$

where

- $N$  is the number of training observations
- $n$  is the number of input features

Output corresponding values (supervised learning)

$$\mathbf{t} = \begin{pmatrix} t^1 \\ \dots \\ t^N \end{pmatrix}$$

where

- $t^i \in \mathbb{R}, \forall i \in \{1 \dots N\} \rightarrow$  regression
- $t^i \in \{C_1 \dots C_K\}, \forall i \in \{1 \dots N\} \rightarrow$  Classification

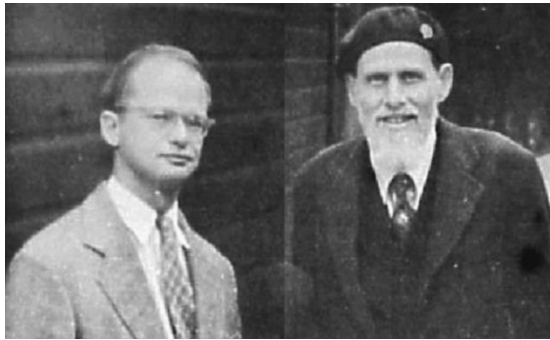
Note that the output dimensionality could be  $> 1$ . In this case,  $M$  stands for this dimension.

- Sometimes, the training set will be rewritten as a set of tuples of input vectors and output values

$$D_N = \{(\mathbf{x}^i, t^i)\}_{1 \leq i \leq N} \text{ where } \mathbf{x}^i \in \mathbb{R}^n$$

- Bold values :
  - uppercase (e.g.,  $\mathbf{X}$ )  $\rightarrow$  matrix
  - lowercase (e.g.,  $\mathbf{x}^i$ )  $\rightarrow$  vector

# HISTORY: A first ARTIFICIAL NEURAL NETWORK



McCulloch &  
Pitts 1943

## A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY

WARREN S. MCCULLOCH and WALTER H. PITTS

Because of the "all-or-none" character of nervous activity, neural events and the relations among them can be treated by means of propositional logic. It is found that the behavior of every net can be described in these terms, with the addition of more complicated logical means for nets containing circles; and that for any logical expression satisfying certain conditions, one can find a net behaving in the fashion it describes. It is shown that many particular choices among possible neurophysiological assumptions are equivalent, in the sense that for every net behaving under one assumption, there exists another net which behaves under the other and gives the same results, although perhaps not in the same time. Various applications of the calculus are discussed.

Bulletin of Mathematical Biophysics; 1943

This paper provided a way to describe brain functions in abstract terms and showed that simple elements connected in a neural network can have immense computational power.



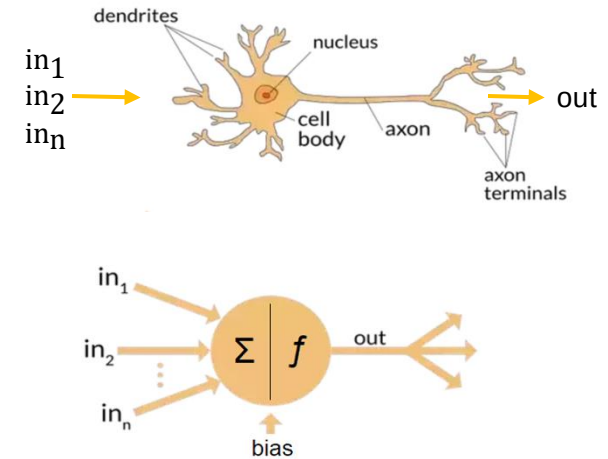
Frank  
Rosenblatt  
1958

## THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN<sup>1</sup>

F. ROSENBLATT

*Cornell Aeronautical Laboratory*

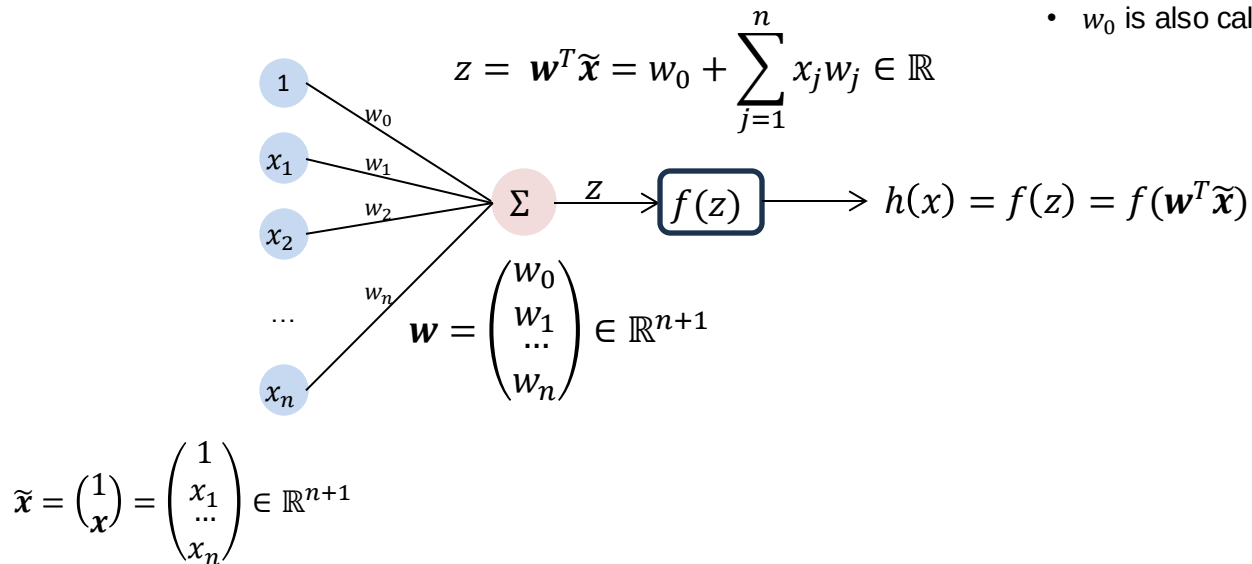
Psychological Review; 1958



# An artificial neuron

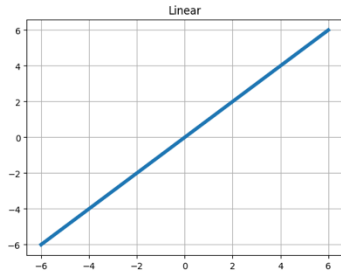
*Vocabulary:*

- $f()$  is the **activation function** of the neuron.
- $(w_0, w_1, \dots, w_n)^T$  are the **synaptic weights**.
- $w_0$  is also called the **threshold** (Bias value).

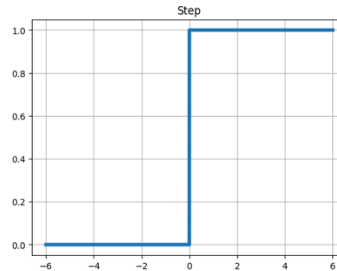


# Activation function $f()$

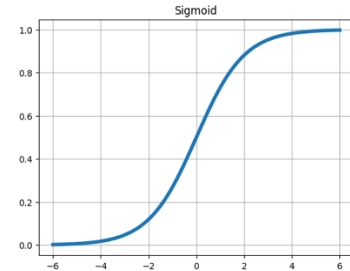
Linear function:  $f(z) = z$



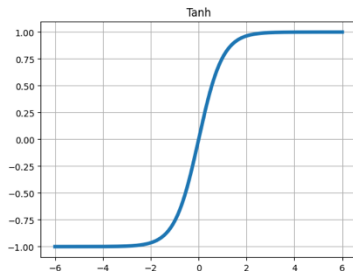
Step function:  $f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$



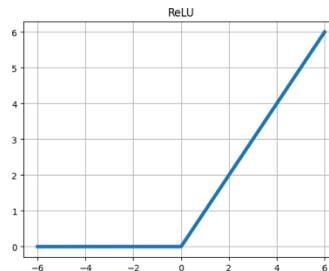
Sigmoid:  $f(z) = \frac{1}{1+e^{-z}}$



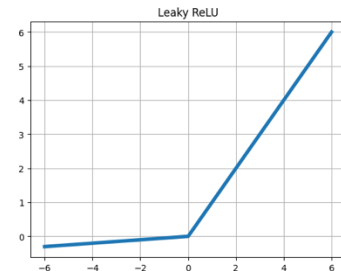
Hyperbolic tangent ( $\tanh$ )



Rectified linear unit (ReLU):  
 $f(z) = \max(0, x)$



Leaky ReLU:  
 $f(z) = \max(\alpha x, x)$





# Loss function

- Let  $J(\mathbf{w}, D_N)$  be the loss function

We will see later other loss functions



- It can be defined **in regression** as follow (*mean squared error (MSE)*):

$$J(\mathbf{w}, D_N) = \frac{1}{N} \sum_{i=1}^N (h(\mathbf{x}^i) - t^i)^2$$

Note: the fraction  $\frac{1}{N}$  is often omitted or replaced by  $\frac{1}{2}$

Given a dataset  $D_N$ , the loss function is defined as a function that depends on the parameters  $\mathbf{w}$ .

Learning: Given a dataset  $D_N$ , search the optimal parameters  $\mathbf{w}$  minimizing  $J(\mathbf{w}, D_N)$ .

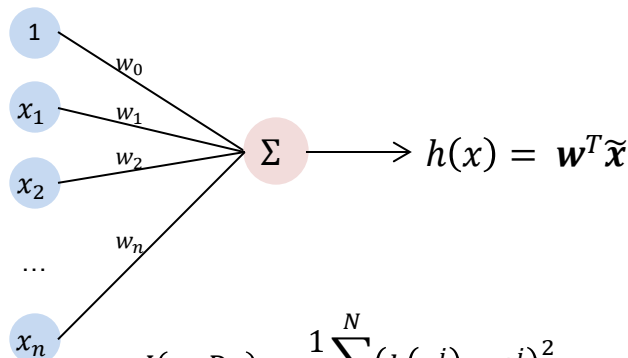
## Gradient of the loss function

- The gradient of  $J(\mathbf{w}, D_N)$  with respect to  $\mathbf{w}$  is denoted as

$$\nabla_{\mathbf{w}} J = \left( \frac{\partial J}{\partial \mathbf{w}} \right) = \left( \frac{\partial J}{\partial w_0}, \frac{\partial J}{\partial w_1}, \dots, \frac{\partial J}{\partial w_n} \right)^T \in \mathbb{R}^{n+1}$$

- In the space of parameters  $\mathbf{w}$ , the gradient vector  $\nabla_{\mathbf{w}} J$  points in the **local direction of the steepest ascent** of the loss function.
- Learning consist of finding the  $\mathbf{w}$  minimizing  $J(\mathbf{w}, D_N)$ .
- In learning, we use the negative gradient,  $-\nabla_{\mathbf{w}} J$ , to update the parameters in the direction that minimizes the loss function.

## Example with a very simple artificial neuron (with $f(z) = z$ )



$$J(\mathbf{w}, D_N) = \frac{1}{2} \sum_{i=1}^N (h(x^i) - t^i)^2$$

$$= \frac{1}{2} \sum_{i=1}^N (\mathbf{w}^T \tilde{\mathbf{x}}^i - t^i)^2 = \frac{1}{2} \|\tilde{\mathbf{X}}\mathbf{w} - \mathbf{t}\|^2$$

where  $\tilde{\mathbf{X}} = \begin{pmatrix} 1 & \mathbf{X} \\ \vdots & \end{pmatrix} \in \mathbb{R}^{N \times (n+1)}$

$$\nabla_{\mathbf{w}} J = \left( \frac{\partial J}{\partial \mathbf{w}} \right) = \left( \frac{\partial J}{\partial w_0}, \frac{\partial J}{\partial w_1}, \dots, \frac{\partial J}{\partial w_n} \right)^T \in \mathbb{R}^{n+1}$$

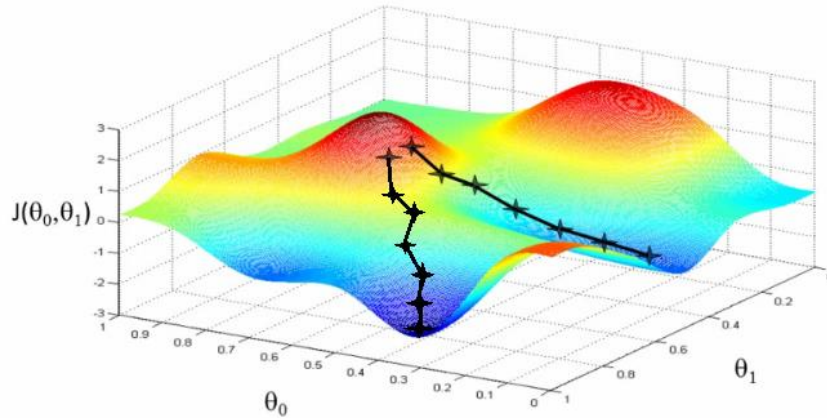
$$\frac{\partial J}{\partial w_j} = \frac{\partial}{\partial w_j} \left( \frac{1}{2} \|\tilde{\mathbf{X}}\mathbf{w} - \mathbf{t}\|^2 \right) = \frac{2}{2} (\tilde{\mathbf{X}}\mathbf{w} - \mathbf{t})^T \left( \frac{\partial}{\partial w_j} \tilde{\mathbf{X}}\mathbf{w} \right) = (\tilde{\mathbf{X}}\mathbf{w} - \mathbf{t})^T \mathbf{x}_j$$

$$\nabla_{\mathbf{w}} J = \left( (\tilde{\mathbf{X}}\mathbf{w} - \mathbf{t})^T \tilde{\mathbf{X}} \right)^T = \tilde{\mathbf{X}}^T (\tilde{\mathbf{X}}\mathbf{w} - \mathbf{t})$$

$$\frac{\partial}{\partial w_j} \tilde{\mathbf{X}}\mathbf{w} = \frac{\partial}{\partial w_j} \begin{pmatrix} 1x_1^1 & \dots & x_n^1 \\ \vdots & \ddots & \vdots \\ 1x_1^N & \dots & x_n^N \end{pmatrix} \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{pmatrix} = \frac{\partial}{\partial w_j} \begin{pmatrix} w_0 + x_1^1 w_1 + \dots + x_j^1 w_j + \dots + x_n^1 w_n \\ \vdots \\ w_0 + x_1^N w_1 + \dots + x_j^N w_j + \dots + x_n^N w_n \end{pmatrix} = \begin{pmatrix} x_j^1 \\ x_j^2 \\ \vdots \\ x_j^N \end{pmatrix}$$

## Batch gradient descent (BGD)

General form:  $\mathbf{w}(t + 1) = \mathbf{w}(t) - \alpha \frac{\partial J}{\partial \mathbf{w}} |_{\mathbf{w}(t)}$



- The value of the gradient  $\nabla_{\mathbf{w}} J$  at  $\mathbf{w}(t)$  gives the *local* direction of the steepest *increasing* slope.
- At each step  $t$ , we move in the *opposite* direction of the gradient.
- The parameter  $\alpha$  (*learning rate*) is used to adapt the step sizes when moving.

## Example with a very simple artificial neuron ( $h(x) = \mathbf{w}^T \tilde{\mathbf{x}}$ )

$$\mathbf{w}(t+1) = \mathbf{w}(t) - \alpha \frac{\partial J}{\partial \mathbf{w}} \big|_{\mathbf{w}(t)}$$

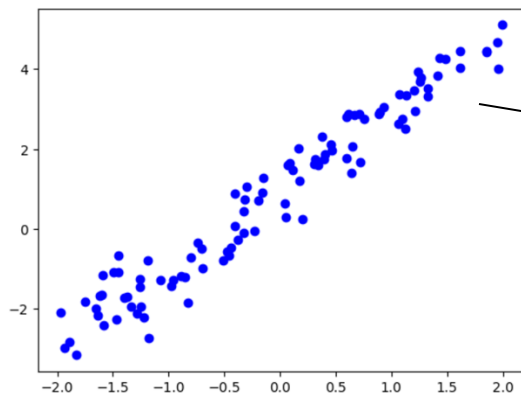


$$\frac{\partial J}{\partial \mathbf{w}} = \tilde{\mathbf{X}}^T (\tilde{\mathbf{X}} \mathbf{w} - \mathbf{t})$$

$$\mathbf{w}(t+1) = \mathbf{w}(t) - \alpha \tilde{\mathbf{X}}^T (\tilde{\mathbf{X}} \mathbf{w}(t) - \mathbf{t})$$

To compute  $\frac{\partial J}{\partial \mathbf{w}}$ , we need the complete dataset  $D_N$  ... and  $D_N$  can be huge!  $\rightarrow$  stochastic gradient descent (SGD)

When  $h(x) = \mathbf{w}^T \tilde{\mathbf{x}}$ , the loss function according the parameters is convex

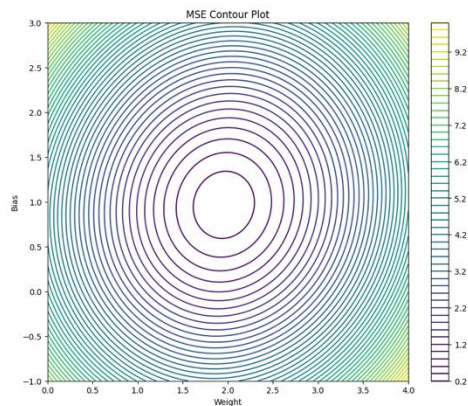
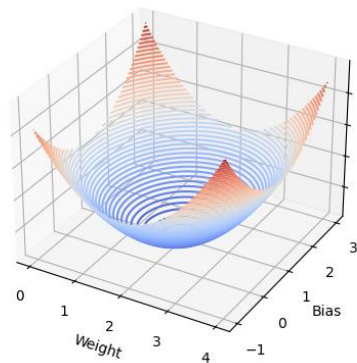
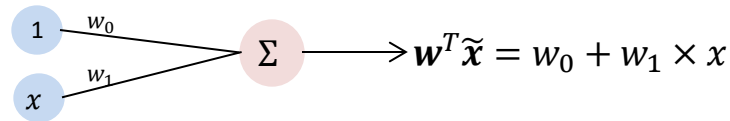


Training data

```
nb_obs = 100
x = torch.tensor(np.random.uniform(-2, 2, nb_obs), dtype=torch.float32)
f = 2*x + 1
y = f + torch.tensor(np.random.normal(0, 0.5, nb_obs), dtype=torch.float32)
```

```
m = nn.Linear(1, 1)
criterion = nn.MSELoss()
```

$$MSE = \sum_{i=1}^N ((w_0 + w_1 \times x^i) - t^i)^2$$



# Training: one sample stochastic gradient descent (1-sample SGD)

If data are *stationery*, **then** minimizing  $J$  is equivalent to successively minimizing each  $J^k$

$$J(\mathbf{w}, D_N) = \frac{1}{2} \sum_{i=1}^N (h(\mathbf{x}^i) - t^i)^2 = \frac{1}{2} \sum_{i=1}^N J^i$$

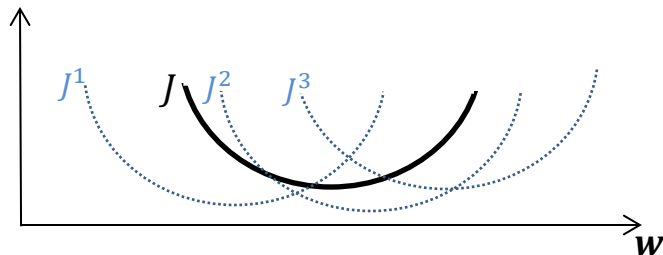
The formula for the update of linear regression models under MSE in the (stochastic) gradient descent becomes:

$$\mathbf{w}(t+1) = \mathbf{w}(t) - \alpha (\mathbf{w}(t)^T \tilde{\mathbf{x}}^k - t^k) \tilde{\mathbf{x}}^k$$

Only one observation is used at each step  $k$ .

Difference between  $i, k$  :

- $i$  and  $k$  are indices on the observations ( $1 \dots N$ )
- $i$  is the index in the database of observations, while  $k$  identifies the order of presentation, which may differ. The difference between  $i$  and  $k$  is not crucial here.



## 1-sample stochastic gradient descent

Initializes  $\mathbf{w}(0)$

Repeat

    Select  $(\mathbf{x}^k, t^k)$  from  $D_N$

    - Forward pass

        Compute output

$$h(\mathbf{x}^k) = \mathbf{w}(t)^T \tilde{\mathbf{x}}^k$$

    - Backward pass

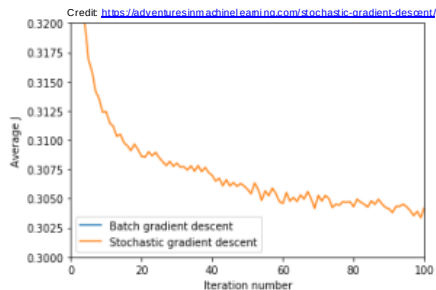
        Compute gradient

$$\nabla_{\mathbf{w}} J^k = (h(\mathbf{x}^k) - t^k) \tilde{\mathbf{x}}^k$$

    - Update parameters

$$\mathbf{w}(t+1) = \mathbf{w}(t) - \alpha \nabla_{\mathbf{w}} J^k$$

# Training: Mini-batch gradient descent



Note about 1-sample SGD: As we are only considering *one* example at a time, the cost will fluctuate based on the training examples and it will not necessarily decrease. But in the long run, you will see the cost decrease with the fluctuations.



Mini-batch gradient descent is a trade-off between 1-sample SGD and the BGD.

In mini-batch gradient descent, the cost function (and therefore gradient) is averaged over a small number of samples (e.g., 10 - 500).

This is opposed to the SGD batch size of *1* sample, and the BGD size of *all* the training samples.

The default **batch\_size** in Keras is 32

Note: in many sources, the terms *mini-batch* and *stochastic gradient descent* are synonymous.



## Question for you!

Neural networks are rarely optimized using Batch Gradient Descent (BGD), and instead we use mini-batches from the dataset to train on (often implemented as Stochastic Gradient Descent, or SGD). What are the advantages of this?

Select all that apply.

- A. BGD is difficult to use on large datasets because we have to store the entire dataset in memory before performing a parameter update.
- B. SGD is able to jump out of local minima of the loss function that would otherwise trap BGD.
- C. SGD gives a better approximation to the true gradient.

# Linear regression with TensorFlow (Keras)

[https://github.com/ocaelen/intro\\_deeplearning/blob/main/tf\\_simple\\_lin\\_reg.ipynb](https://github.com/ocaelen/intro_deeplearning/blob/main/tf_simple_lin_reg.ipynb)

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers.experimental import preprocessing
```

```
In [2]: x = np.linspace(-1,1, 200)
y = 3*x + 2 + np.random.normal(size = len(x),scale=0.3)
x = x.reshape((-1,1))
y = y.reshape((-1,1))
```

```
In [3]: model = Sequential([
    Dense(1,input_shape=(1,)) # Default activation: 'linear'
])
model.summary()
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
hist = model.fit(x, y, epochs=1500, verbose=0)
```

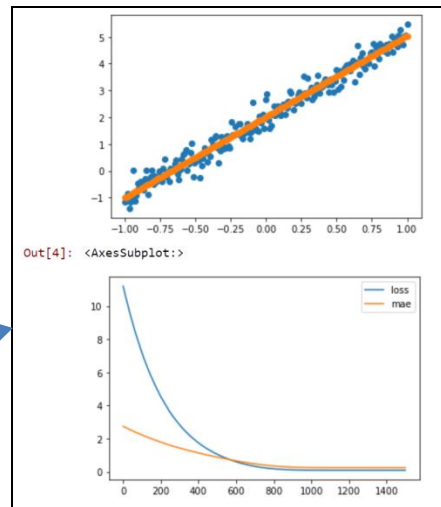
Model: "sequential"

| Layer (type)            | Output Shape | Param # |
|-------------------------|--------------|---------|
| dense (Dense)           | (None, 1)    | 2       |
| Total params: 2         |              |         |
| Trainable params: 2     |              |         |
| Non-trainable params: 0 |              |         |

```
In [4]: y_hat=model.predict(x)

plt.scatter(x,y)
plt.scatter(x,y_hat)
plt.show()

pd.DataFrame(hist.history).plot()
```



# Linear regression with PyTorch

```
import torch
import torch.nn as nn
import numpy as np
from sklearn import datasets
from pickletools import optimize
import matplotlib.pyplot as plt
```

```
## Prepare data
X_numpy, y_numpy = datasets.make_regression(n_samples=200, n_features=1, noise=10, random_state=42)
X = torch.from_numpy(X_numpy.astype(np.float32))
y = torch.from_numpy(y_numpy.astype(np.float32))
y = y.view(y.shape[0], 1)

n_samples, n_features = X.shape
```

```
m = nn.Linear(n_features , 1) # input_size=1, output_size=1
```

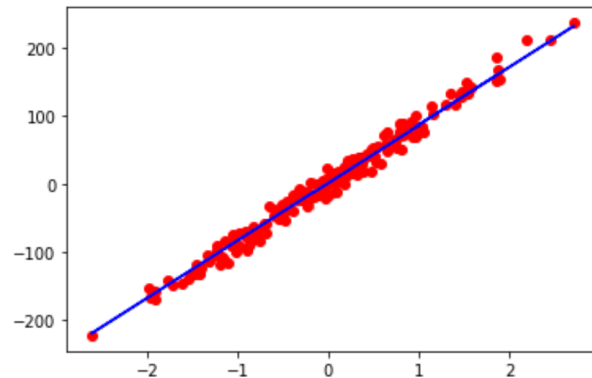
```
loss_fct = nn.MSELoss()
optimizer = torch.optim.SGD(m.parameters(), lr=0.01)
```

```
for epoch in range(200):
    y_pred = m(X) # forward pass
    loss = loss_fct(y_pred, y)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()

    if (epoch+1) % 40 == 0:
        print(f'epoch: {epoch+1}, loss:{loss.item():.4f}')
```

```
epoch: 40, loss:1757.2028
epoch: 80, loss:514.2151
epoch: 120, loss:204.7480
epoch: 160, loss:127.4618
epoch: 200, loss:108.1133
```

```
predicted = m(X).detach().numpy()
plt.plot(X_numpy, y_numpy, 'ro')
plt.plot(X_numpy, predicted, 'b')
plt.show()
```



# Logistic regression with TensorFlow (Keras)

```
In [1]: 1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 from sklearn.preprocessing import StandardScaler
4 from tensorflow.keras import Sequential
5 from tensorflow.keras.layers import Dense
6 import pandas as pd
```

```
In [2]: 1 df = sns.load_dataset('iris')
2
3 ## Delete a class to have a binary classification problem
4 df = df[df['species'] != 'setosa']
```

```
In [3]: 1 sns.pairplot(df, kind="scatter", hue="species", plot_kws=dict(s=80, edgecolor="white", linewidth=2.5))
2 plt.show()
```

```
In [4]: 1 X = df.iloc[:,0:4].values
2 X = StandardScaler().fit_transform(X)
3
4 y = (df.iloc[:,4].values == 'versicolor').astype(int)
5
6 print(X[:5,:])
7 print(y[:5])
```

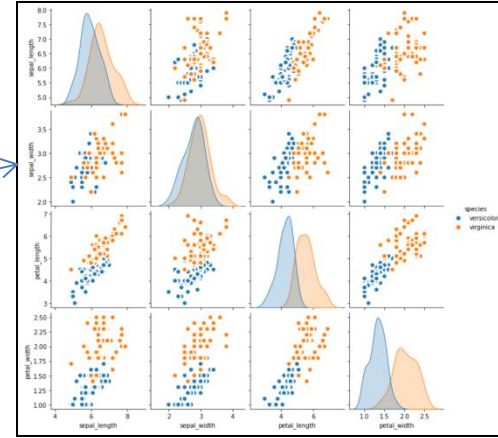
```
In [5]: 1 model = Sequential([
2     Dense(1,input_shape=(4,), activation='sigmoid') # Default activation: 'Linear'
3 ])
4 model.summary()
5 model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
6 hist = model.fit(X, y, epochs=1500, verbose=0)
```

Model: "sequential"

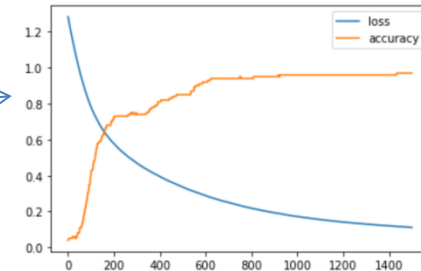
| Layer (type)          | Output Shape | Param # |
|-----------------------|--------------|---------|
| dense (Dense)         | (None, 1)    | 5       |
| Total params:         | 5            |         |
| Trainable params:     | 5            |         |
| Non-trainable params: | 0            |         |

```
In [6]: 1 pd.DataFrame(hist.history).plot()
2 test_loss, test_accuracy=model.evaluate(X,y, verbose=0)
3 print(f'Test loss: {test_loss}')
4 print(f'Test accuracy: {test_accuracy}')
```

Test loss: 0.11126507073640823  
Test accuracy: 0.9700000286102295



```
[[ 1.11900931  0.99068792 -0.25077906 -0.65303909]
 [ 0.20924564  0.99068792 -0.49425387 -0.41643072]
 [ 0.96738203  0.68864892 -0.00730424 -0.41643072]
 [-1.15539985 -1.72766308 -1.10294091 -0.88964745]
 [ 0.36087292 -0.21746808 -0.37251647 -0.41643072]]
[1 1 1 1 1]
```



## **Multilayer perceptron**

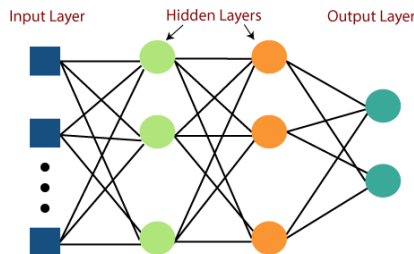
# Multilayer perceptron (MLP)

(also called *Feed-forward neural networks* or *Plain vanilla*)

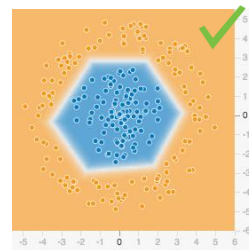
- Combine input information in a complex & flexible neural net model

- Network Structure

- Multiple layers
  - Input layer
  - Hidden layer(s)
  - Output layer



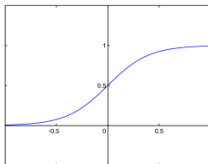
Warning: this representation don't show the bias values!



- Multilayer perceptron → fully connected layers

- Activation functions  $f$  in the hidden layers introduces nonlinearity.

- Historically, the most common activation functions in the hidden layers was sigmoid.



$$f(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

← Old school! For hidden layers in MLP use ReLu

$\sigma(z)$  transform any input  $z \in \mathbb{R}$  into a number in  $[0, 1]$ .

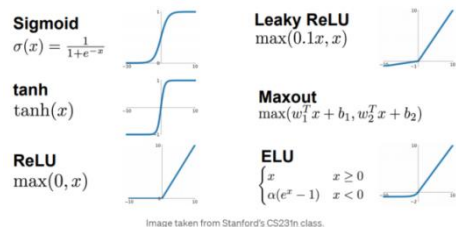
## Question for you!

Why is it needed to add non linear activation function?

What if all activation functions are linear?

# Activation function

Some other popular examples of activation functions



But... How to choose the activation function that we will use in our deep learning model ??

There are no general rules but some *common* rules depending on the type of layers (input layer, hidden layer(s) and output layer)

→ For the input layer, it's simple: As input layers takes raw input from the domain, there are no activation functions in the input layer.

## Activation for Hidden Layers

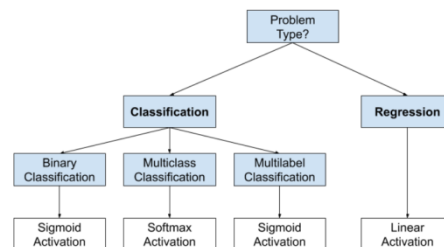
There might be three activation functions that you might want to consider using in hidden layers; they are:

- Logistic (Sigmoid)
  - Traditionally, the sigmoid activation function was the default activation function in the 1990s.
  - Still used in some advance cases (e.g., recurrent neural network)
- Hyperbolic Tangent (Tanh)
  - Still used in some advance cases (e.g., recurrent neural network)
- Rectified Linear Activation (ReLU)
  - *"In modern neural networks, the default recommendation is to use the rectified linear unit or ReLU (...)"* I. Goodfellow et al. *Deep Learning*, 2016
  - nowadays (2025) we tend to replace ReLU by LeakyReLU

An MLP neural network will almost always have the same activation function in all hidden layers.

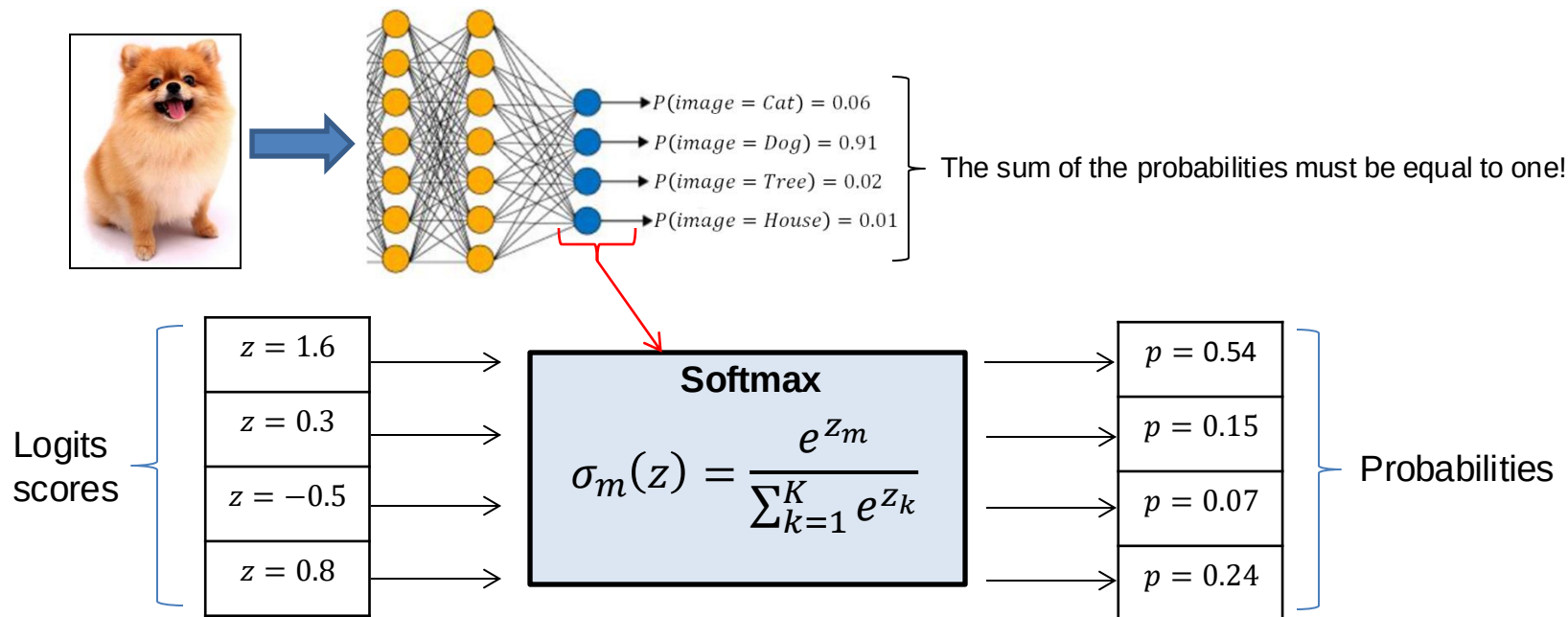
## Activation for Output Layers

How to Choose an Output Layer Activation Function



# Activation function – Softmax

- **Softmax** is a kind of generalization of the logistic function for the multiclass classification problem.
- In a multiclass classification problem:
  - the target variable can take many distinct discrete values :  $t^i \in \{C_1 \dots C_K\}$  where  $K$  is the number of class.
  - The output of the model  $h(x)$  is a vector of values in  $[0,1]$  which sum to 1 (i.e. probabilities)





## Homework [optional!]

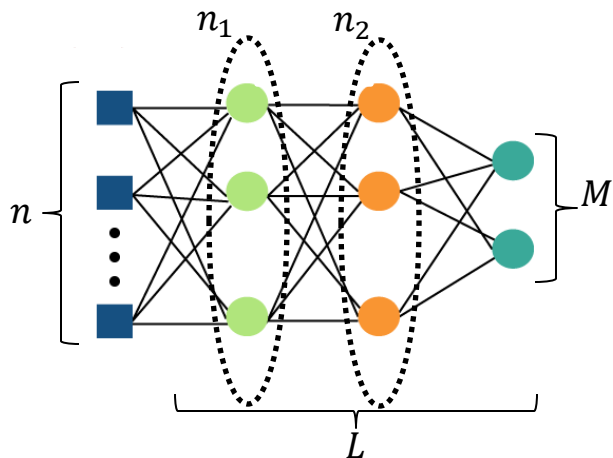
### Youtube

- **3Blue1Brown** But what is a Neural Network? | Deep learning, chapter 1  
<https://www.youtube.com/watch?v=aircAruvnKk>
- **3Blue1Brown** Gradient descent, how neural networks learn | Deep learning, chapter 2  
<https://www.youtube.com/watch?v=IHZwWFHWa-w>

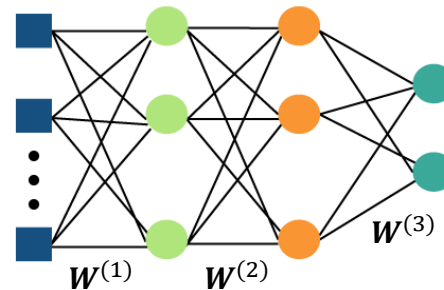
Please send a summary of the two videos:

- What did you learn as new concepts that are not in the slides?
- One A4 page max.
- Deadline: in 3 weeks
- One extra point (+1/20) in addition to the oral assessment
- Warning: if you do the work, to have the (+1/20), you may have a question about the videos during the exams to check that you have indeed seen the video.

# Multilayer perceptron – notation



- $n$  : number of input features.
- $n_1, n_2, \dots, n_l, \dots$  : number of nodes in layers
- $M$  : number of output values
- $L$  : number of layers.



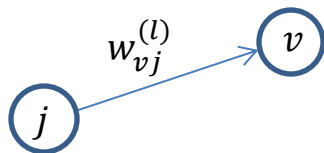
$\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times (n_{l-1}+1)}$ : synaptic weights between layers  $(l - 1)$  and  $l$ .

Example, if  $n = 1$  and  $n_1 = 3$  then

$$\mathbf{W}^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} \end{pmatrix} \in \mathbb{R}^{3 \times 2}$$

## Multilayer perceptron – notation

Let  $w_{vj}^{(l)}$  be the synaptic weight between neuron  $j$  and neuron  $v$  in layer  $l$ .



$$\text{with: } \mathbf{W}^{(l)} = \begin{pmatrix} w_{1,1}^{(l)} & \cdots & w_{1,(n_{l-1}+1)}^{(l)} \\ \vdots & w_{v,j}^{(l)} & \vdots \\ w_{n_l,1}^{(l)} & \cdots & w_{n_l,(n_{l-1}+1)}^{(l)} \end{pmatrix} \in \mathbb{R}^{n_l \times (n_{l-1}+1)}$$

# Hyperparameters

The “structure” of a multilayer perceptron is characterized by many hyperparameter

- number of hidden layers
- number of nodes (neurons) per hidden layer
- activation functions in each layer

These hyperparameters are defined (chosen) by the data scientist.

- Note that hyperparameter identification by ‘cross-validation like’ or ‘grid search like’ methods is difficult with DL --> question for you: why?

Others are constrained by the modeling problem e.g.,

- number of input features
- number of output values
- activation function in the last layer (i.e., regression or classification)

## Expressiveness : One-hidden layer vs many hidden layers (A.k.a. deep learning )

- A one Hidden Layer Neural Network Architecture with sigmoid/hyperbolic units and with enough neurons in its hidden layer (even  $\infty$  neurones if necessary) can approximate arbitrarily closely any function  
-> Universal approximation
- However, as we increase the number of layers, the total number of neurones needed to have the same flexibility can decrease (even exponentially) with the number of layers.

## Questions for you!

Draw the following neural network:

- $L = 3$
- $n = 1$
- $n_1 = 2$
- $n_2 = 3$
- $M = 1$

How many synaptic weights (tips: don't forget the bias values ;- ) )?

- A. 12 synaptic weights
- B. 17 synaptic weights
- C. 49 synaptic weights
- D. 15 synaptic weights

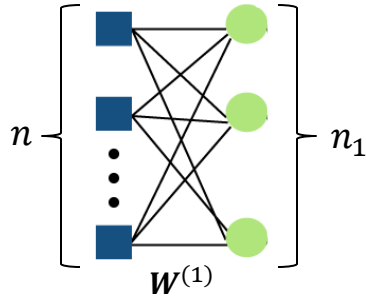
How many trainable parameters does a feedforward network have with input shape (64,), three hidden layers with 16 units each and a final linear layer with 8 units?

- 1720
- 968
- 7416
- 142

Why do we use non-linear activation functions (such as *tanh* or *relu*) in neural networks?

- A. To allow the usage of higher learning rates, thus speeding up the convergence during the optimization.
- B. To induce sparse connectivity in the network weights.
- C. Without non-linear activation functions, the network would only be able to model linear functions of the data.

## Multilayer perceptron – formula

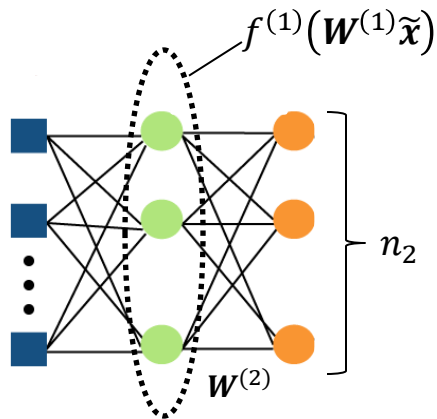


$$\mathbf{W}^{(1)} \in \mathbb{R}^{n_1 \times (n+1)} \rightarrow \mathbf{W}^{(1)} \tilde{\mathbf{x}} \in \mathbb{R}^{n_1 \times 1} \rightarrow f^{(1)}(\mathbf{W}^{(1)} \tilde{\mathbf{x}}) = \mathbf{a}_1 = \begin{pmatrix} a_1^{(1)} \\ \vdots \\ a_{n_1}^{(1)} \end{pmatrix} \in \mathbb{R}^{n_1 \times 1}$$

$$\tilde{\mathbf{x}} \in \mathbb{R}^{(n+1) \times 1}$$

$a_j^{(l)}$  is the output of node  $j$  in layer  $l$

## Multilayer perceptron – formula



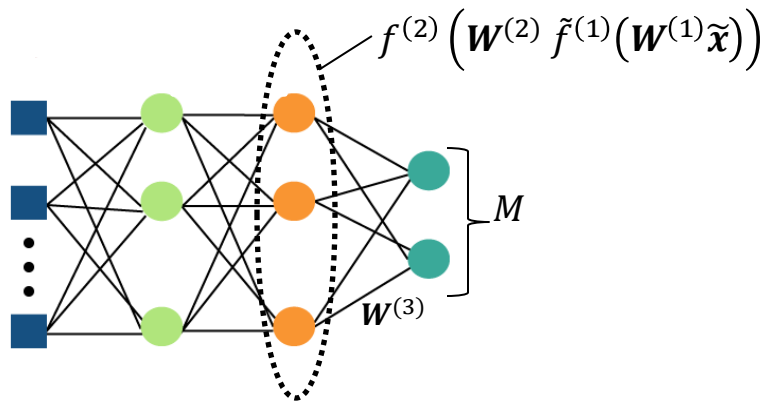
$$W^{(2)} \in \mathbb{R}^{n_2 \times (n_1 + 1)} \quad \rightarrow \quad W^{(2)} \tilde{f}^{(1)}(W^{(1)}\tilde{x}) \in \mathbb{R}^{n_2 \times 1} \quad \rightarrow \quad f^{(2)}\left(W^{(2)} \tilde{f}^{(1)}(W^{(1)}\tilde{x})\right) = \begin{pmatrix} a_1^{(2)} \\ \vdots \\ a_{n_2}^{(2)} \end{pmatrix} \in \mathbb{R}^{n_2 \times 1}$$

$$f^{(1)}(W^{(1)}\tilde{x}) \in \mathbb{R}^{n_1 \times 1}$$

where  $\tilde{f}^{(1)}(W^{(1)}\tilde{x}) = \begin{pmatrix} 1 \\ f^{(1)}(W^{(1)}\tilde{x}) \end{pmatrix} \in \mathbb{R}^{(n_1+1) \times 1}$



## Multilayer perceptron – formula



$$W^{(3)} \in \mathbb{R}^{M \times (n_2+1)}$$

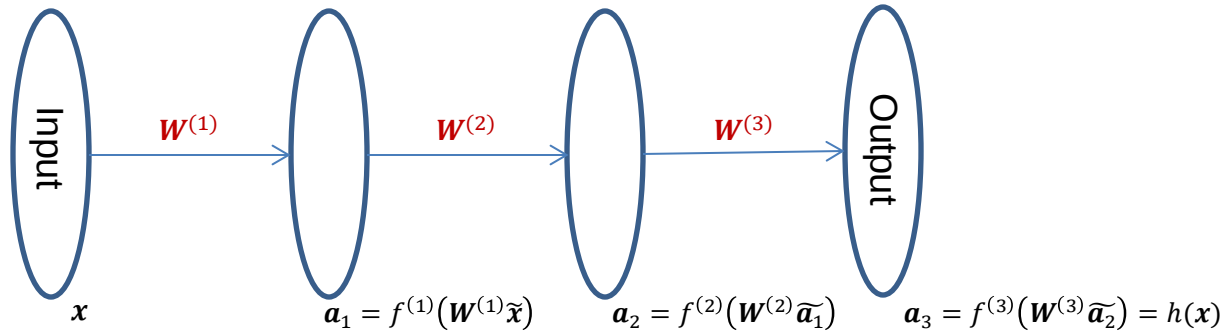
$$f^{(2)}(W^{(2)} \tilde{f}^{(1)}(W^{(1)} \tilde{x})) \in \mathbb{R}^{n_2 \times 1}$$



$$h(x) = f^{(3)}(W^{(3)} \tilde{f}^{(2)}(W^{(2)} \tilde{f}^{(1)}(W^{(1)} \tilde{x}))) \in \mathbb{R}^{M \times 1}$$

where  $\tilde{f}^{(2)}(W^{(2)} \tilde{f}^{(1)}(W^{(1)} \tilde{x})) = \begin{pmatrix} 1 \\ f^{(2)}(W^{(2)} \tilde{f}^{(1)}(W^{(1)} \tilde{x})) \end{pmatrix} \in \mathbb{R}^{(n_2+1) \times 1}$

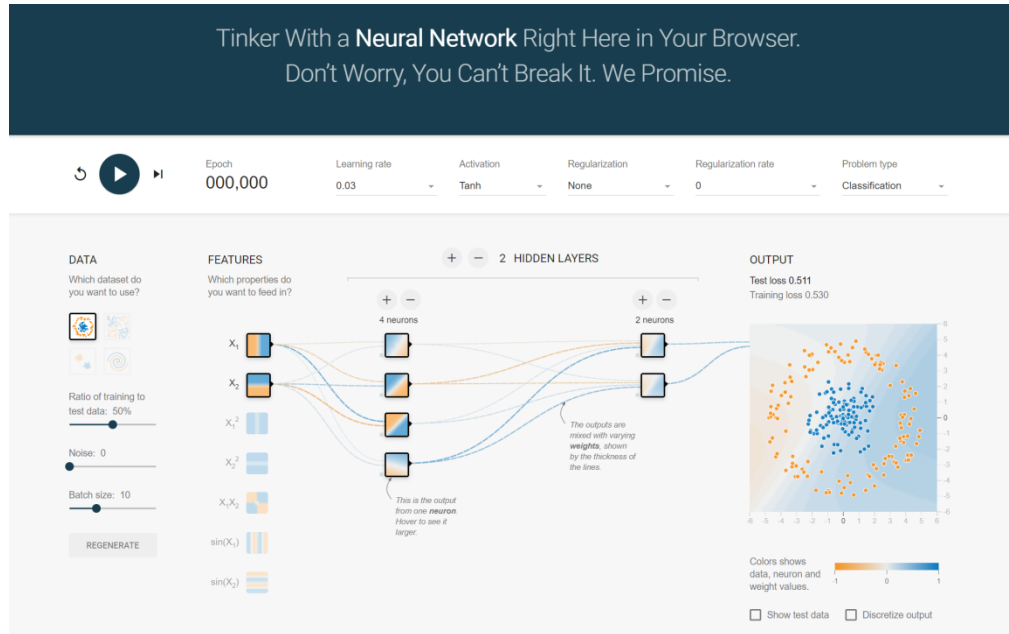
## Multilayer perceptron – formula



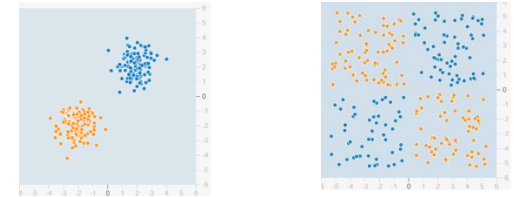
$$h(x) = f^{(3)}\left(W^{(3)}\tilde{f}^{(2)}\left(W^{(2)}\tilde{f}^{(1)}\left(W^{(1)}\tilde{x}\right)\right)\right)$$

# Demo: Tensorflow playground

<https://playground.tensorflow.org/>



- What is the simplest architecture for these two learning problems?



- Make a DL model with max layers & max nodes (cf. gradient vanishing issue)
  - test sigmoid activation function
  - test ReLu activation function
- Question: why on the *same problem* with *the same architecture* do we get different results?

## Back to the loss function $J$

- The loss function  $J$  is used to evaluate the error of the model for a given input.
- It depends on a model (represented by the weights  $\mathbf{W}$ ) and a set of input/output samples  $D_N$ 
  - $J(\mathbf{W}, D_N)$
- Many functions can be used,
  - in regression: *mean squared error* (MSE) is the most used
    - $J = \frac{1}{N} \sum_{i=1}^N (h(\mathbf{x}^i) - t^i)^2 = \frac{1}{N} \sum_{i=1}^N J^i$  where  $J^i$  is the error on the sample  $(\mathbf{x}^i, t^i)$ .
  - in classification: the cross-entropy loss is the most used
    - Although technically the MSE can also be used (less strict on errors, poorer performance)

## Binary cross-entropy - Commonly used by neural networks for binary-classification problems

Let

- $\{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$  be a dataset with N observations
- $h(x_i) \in [0, 1]$  be the output of the model
- $y_i \in \{0, 1\}$  be the true output



**Binary cross-entropy:**

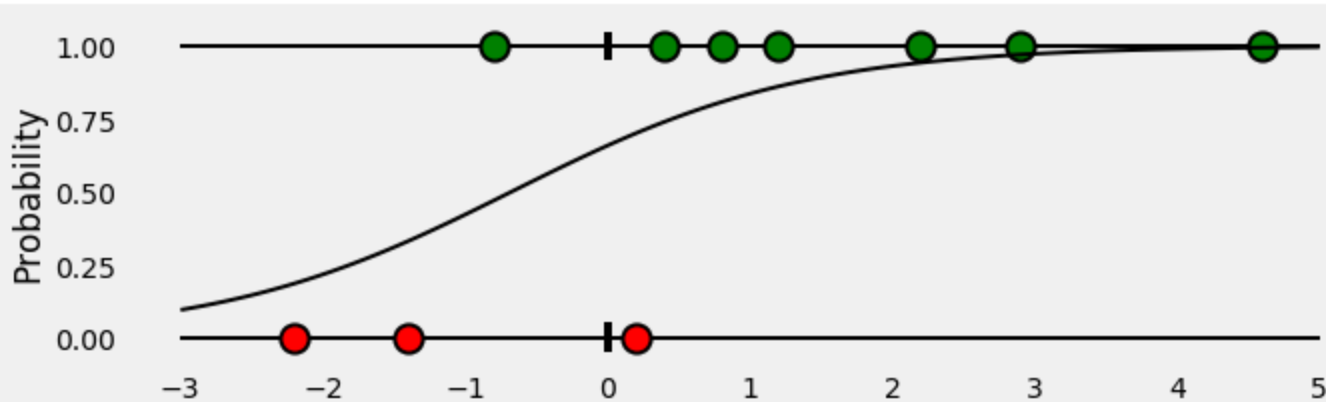
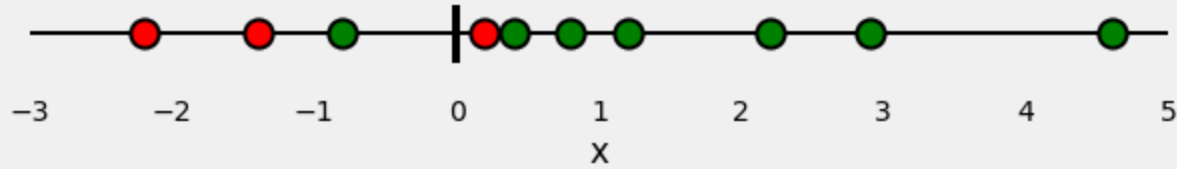
$$CE(h) = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(h(x_i)) + (1 - y_i) \cdot \log(1 - h(x_i))$$

Reading this formula:

- For each "y=1", it adds  $-1 \times \log(h(x_i))$  to the loss.
- For each "y=0", it adds  $-1 \times \log(1 - h(x_i))$  to the loss

# Binary cross-entropy — the visual way (1)

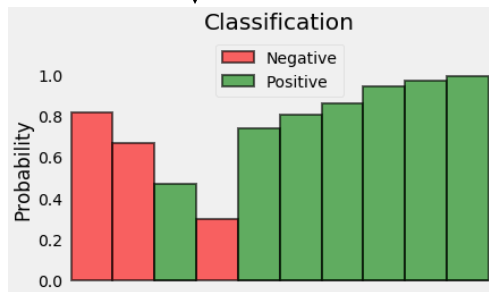
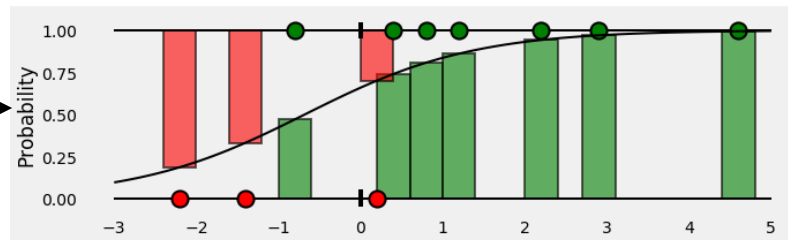
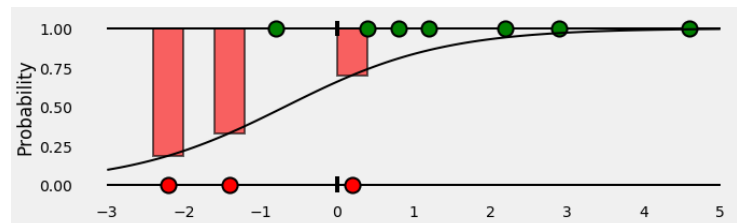
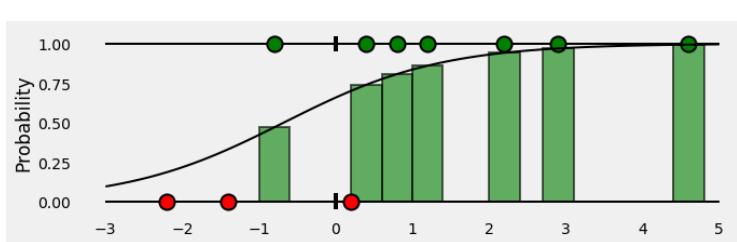
Credit : <https://towardsdatascience.com/understanding-binary-cross-entropy-log-loss-a-visual-explanation-a3ac6025181a>



Logistic regression model

# Binary cross-entropy — the visual way (2)

Credit : <https://towardsdatascience.com/understanding-binary-cross-entropy-log-loss-a-visual-explanation-a3ac6025181a>



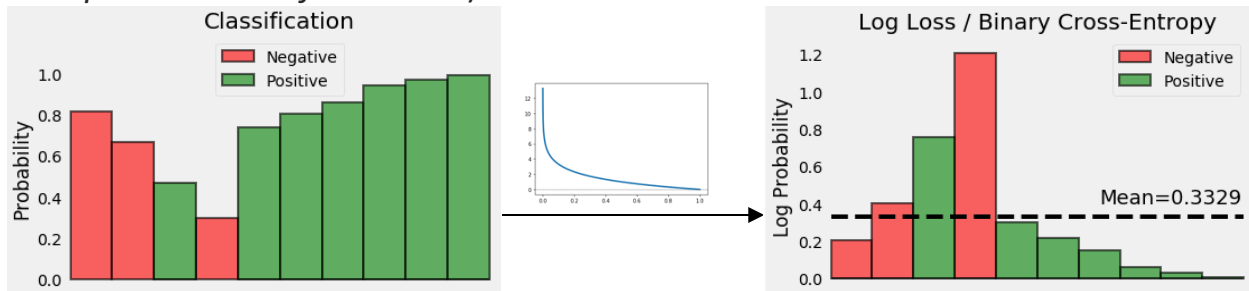
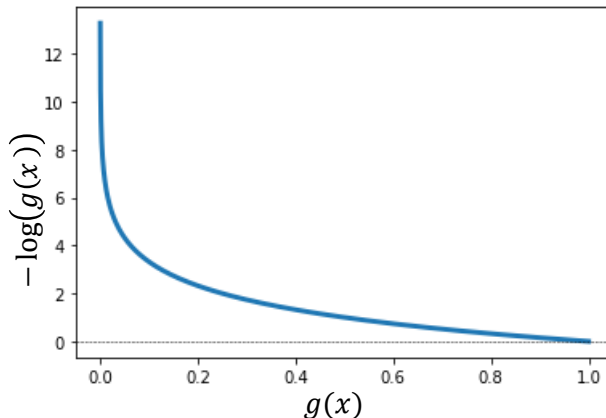
If the **probability** associated with the **true class** is **1.0**, we need its **loss** to be **zero**. Conversely, if that **probability** is **low**, say **0.01**, we need its **loss** to be **HUGE**!

## Binary cross-entropy — the visual way (3)

If the **probability** associated with the **true class** is **1.0**, we need its **loss** to be **zero**.

Conversely, if that **probability is low**, say **0.01**, we need its **loss** to be **HUGE**!

It turns out, taking the **negative log of the probability** suits us well enough for this purpose (*since the log of values between 0.0 and 1.0 is negative, we take the negative log to obtain a positive value for the loss*).



Binary cross-entropy:

$$CE(h) = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(h(x_i)) + (1 - y_i) \cdot \log(1 - h(x_i))$$

Finally, we compute the **mean of all these losses**.

The **binary cross-entropy** of this example is **0.3329**.



## Multi-class cross-entropy loss - Commonly used by neural networks for multi-class problems

$$CE(h_y(x_i), y_i) = - \sum_i y_i \log(h_y(x_i))$$



|                            | $g_y(x_i)$ | LABELS |
|----------------------------|------------|--------|
| $P(\text{Dog}   x_i)$      | 0.80       | 1.00   |
| $P(\text{Cat}   x_i)$      | 0.03       | 0.00   |
| $P(\text{Fox}   x_i)$      | 0.03       | 0.00   |
| $P(\text{Elephant}   x_i)$ | 0.02       | 0.00   |
| $P(\text{Lion}   x_i)$     | 0.01       | 0.00   |
| $P(\text{Mouse}   x_i)$    | 0.00       | 0.00   |
| $P(\text{Bird}   x_i)$     | 0.03       | 0.00   |
| $P(\text{Pokémon}   x_i)$  | 0.04       | 0.00   |
| $P(\text{Fish}   x_i)$     | 0.01       | 0.00   |
| $P(\text{None}   x_i)$     | 0.03       | 0.00   |

| $h_y(x_i)$ |
|------------|
| 0.7        |
| 0.2        |
| 0.1        |
| 0.9        |

| $y_i$ |
|-------|
| 1     |
| 0     |
| 0     |
| 0     |

$$CE(h_y(x_i), y_i) = -\log(0.7) = 0.5145$$

# MLP in TensorFlow

[https://github.com/ocaelen/intro\\_deeplearning/blob/main/many\\_mlp.ipynb](https://github.com/ocaelen/intro_deeplearning/blob/main/many_mlp.ipynb)

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
```

## Binary classification

```
model = Sequential([
    Dense(10, input_shape=(4,), activation='relu'),
    Dense(6, activation='relu'),
    Dense(4, activation='relu'),
    Dense(1, activation='sigmoid')
])
model.summary()
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
hist = model.fit(X, y, epochs=15, verbose=0)
```

Model: "sequential"

| Layer (type)            | Output Shape | Param # |
|-------------------------|--------------|---------|
| =====                   |              |         |
| dense (Dense)           | (None, 10)   | 50      |
| dense_1 (Dense)         | (None, 6)    | 66      |
| dense_2 (Dense)         | (None, 4)    | 28      |
| dense_3 (Dense)         | (None, 1)    | 5       |
| =====                   |              |         |
| Total params: 149       |              |         |
| Trainable params: 149   |              |         |
| Non-trainable params: 0 |              |         |

## Multiclass classification

```
model = Sequential([
    Dense(10, input_shape=(4,), activation='relu'),
    Dense(6, activation='relu'),
    Dense(4, activation='relu'),
    Dense(3, activation='softmax')
])
model.summary()
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
hist = model.fit(X, y, epochs=15, verbose=0)
```

Model: "sequential\_1"

| Layer (type)            | Output Shape | Param # |
|-------------------------|--------------|---------|
| =====                   |              |         |
| dense_4 (Dense)         | (None, 10)   | 50      |
| dense_5 (Dense)         | (None, 6)    | 66      |
| dense_6 (Dense)         | (None, 4)    | 28      |
| dense_7 (Dense)         | (None, 3)    | 15      |
| =====                   |              |         |
| Total params: 159       |              |         |
| Trainable params: 159   |              |         |
| Non-trainable params: 0 |              |         |

## Regression

```
model = Sequential([
    Dense(10, input_shape=(3,), activation='relu'),
    Dense(6, activation='relu'),
    Dense(4, activation='relu'),
    Dense(1) # Default activation: 'linear'
])
model.summary()
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
hist = model.fit(X, y, epochs=15, verbose=0)
```

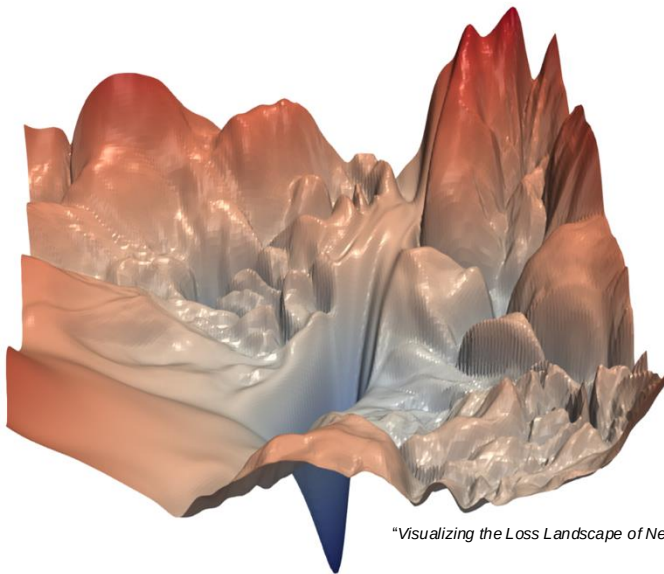
Model: "sequential\_2"

| Layer (type)            | Output Shape | Param # |
|-------------------------|--------------|---------|
| =====                   |              |         |
| dense_8 (Dense)         | (None, 10)   | 40      |
| dense_9 (Dense)         | (None, 6)    | 66      |
| dense_10 (Dense)        | (None, 4)    | 28      |
| dense_11 (Dense)        | (None, 1)    | 5       |
| =====                   |              |         |
| Total params: 139       |              |         |
| Trainable params: 139   |              |         |
| Non-trainable params: 0 |              |         |

# Learning in DL context

Learning: Given a training set  $D_N$ , search the optimal parameters  $\mathbf{w}$  minimizing  $J(\mathbf{w}, D_N)$ .

- Learning is an optimization problem.
  - Therefore, many optimization algorithms can be used to find the best  $\mathbf{w}$ .
- For deep learning, the function  $J(\mathbf{w}, D_N)$  that we have to minimize is too complex to find the global minimum.
  - We need metaheuristic (i.e., find *a good local minimum* rather than the global minimum)



"Visualizing the Loss Landscape of Neural Nets" Nov 2017

# Learning in DL context

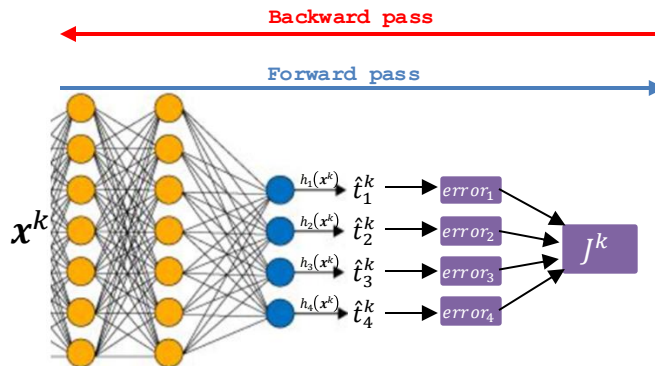
- For deep learning, the most commonly used algorithms are gradient descent algorithms...
  - ... and it's what we will see in this lecture!
- But other metaheuristic *could* be used (e.g., simulated annealing)
  - but it's still very marginal



## Training: Backpropagation

- During the learning process, “coefficients” (synaptic weights & thresholds) are continually updated in an iterative process.
- Gradient descent algorithms are typically used to learn neural network models.
- Backpropagation is a method of calculating the error gradient for each neuron in a neural network, from the last layer to the first.
- *For simplicity*, let us first consider 1-sample stochastic gradient descent (i.e., Only one observation used at each step  $k$ ) and regression.

# Training: Backpropagation



```
Initializes  $w(0)$   
Repeat  
  Select  $(x^k, t^k)$  from  $D_N$ 
```

```
  - Forward pass  
    Compute output  $h(x^k)$ 
```

```
  - Backward pass  
    Compute gradient  $\nabla_w J^k$ 
```

```
  - Update parameters  
     $w(t+1) = w(t) - \alpha \nabla_w J^k$ 
```

Let's focus on this step

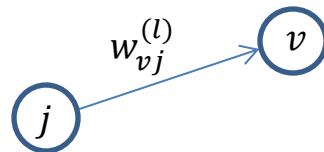
## Training: Backpropagation (compute $\nabla_w J^k$ )

- Let  $J^k$  be the error of  $h(\mathbf{x}^k) \in \mathbb{R}^M$  on the training observation  $(\mathbf{x}^k, \mathbf{t}^k)$  where  $M$  is the output dimension.

- $J^k$  can be defined as follow:  $J^k = \sum_{v=1}^M (t_v^k - h_v(\mathbf{x}^k))^2$ .

- Let  $w_{vj}^{(l)}$  be the synaptic weight between neuron  $j$  and neuron  $v$  in layer  $l$ .

$$\mathbf{W}^{(l)} = \begin{pmatrix} w_{1,1}^{(l)} & \cdots & w_{1,(n_{l-1}+1)}^{(l)} \\ \vdots & w_{v,j}^{(l)} & \vdots \\ w_{n_l,1}^{(l)} & \cdots & w_{n_l,(n_{l-1}+1)}^{(l)} \end{pmatrix} \in \mathbb{R}^{n_l \times (n_{l-1}+1)}$$



- $\nabla_w J^k = \left( \frac{\partial J^k}{\partial w_{1,1}^{(1)}}, \frac{\partial J^k}{\partial w_{1,2}^{(1)}}, \dots, \frac{\partial J^k}{\partial w_{n_L,(n_{L-1}+1)}^{(L)}} \right)^T \rightarrow$  the partial derivative must be computed for all weight  $w_{vj}^{(l)}$ .
  - the backpropagation does it layer after layer starting at the output.

## Reminder – Chain rule

For concreteness, consider the function:  $y(x) = e^{\sin(x^2)}$

$$\text{If } \begin{cases} y = f(u) = e^u \\ u = g(v) = \sin(v) \\ v = h(x) = x^2 \end{cases}, \text{ then the chain rule states that } \frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \frac{\partial u}{\partial v} \frac{\partial v}{\partial x}$$

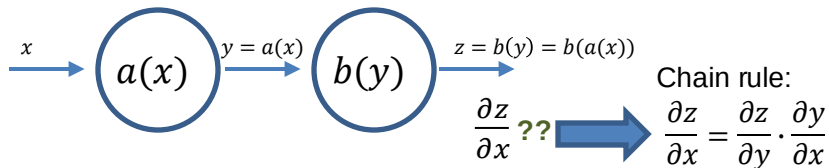
As,

- $\frac{\partial y}{\partial u} = f'(u) = e^u = e^{\sin(v)} = e^{\sin(x^2)}$
- $\frac{\partial u}{\partial v} = g'(v) = \cos(v) = \cos(x^2)$
- $\frac{\partial v}{\partial x} = h'(x) = 2x$

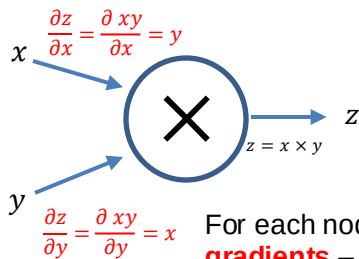
$$\rightarrow \frac{\partial y}{\partial x} = e^{\sin(x^2)} \cos(x^2) 2x$$



# Automatic differentiation & computational graph

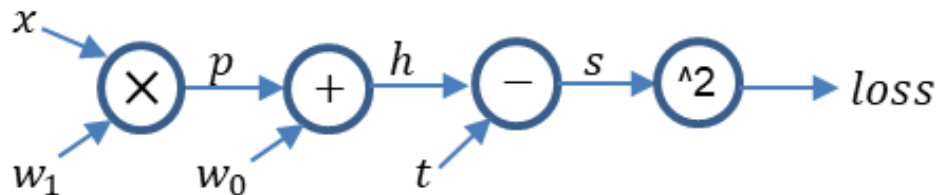


For each operation we perform on tensor, PyTorch and TensorFlow create a graph for us where, at each node, we apply an operation or a function with inputs and obtain outputs.



For each node, we can compute the **local gradients** – that will we used in the chain rule.

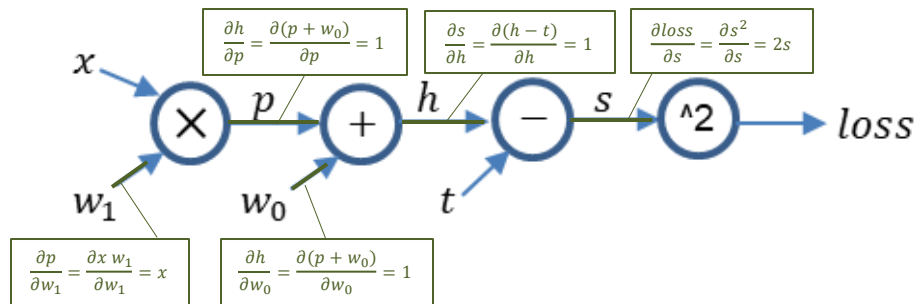
$$h(x) = x w_1 + w_0 \quad \text{and} \quad \text{loss} = (h(x) - t)^2 = (x w_1 + w_0 - t)^2$$



$$\frac{\partial \text{loss}}{\partial w_0} = ?? \quad \text{and} \quad \frac{\partial \text{loss}}{\partial w_1} = ??$$

# Automatic differentiation & computational graph

$$h(x) = x w_1 + w_0 \quad \text{and} \quad \text{loss} = (h(x) - t)^2 = (x w_1 + w_0 - t)^2$$



1. Compute all the **local gradients** in the graph
2. Apply the chain rule

- $\frac{\partial \text{loss}}{\partial w_0} = 2s \times 1 \times 1 = 2s$
- $\frac{\partial \text{loss}}{\partial w_1} = 2s \times 1 \times 1 \times x = 2sx$

Example:  $x = 2$ ,  $w_0 = 3$ ,  $w_1 = 4$ ,  $t = 5$

Forward pass  $\Rightarrow p = 8$ ,  $h = 11$ ,  $s = 6$ ,  $\text{loss} = 36$

$\Rightarrow \frac{\partial \text{loss}}{\partial w_0} = 2 \times 6 = 12$ ,  $\frac{\partial \text{loss}}{\partial w_1} = 2 \times 6 \times 2 = 24$

at  $w = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$ ,  $\nabla_w J = \begin{pmatrix} 12 \\ 24 \end{pmatrix}$

```
import torch
```

```
x = torch.tensor(2.0)
t = torch.tensor(5.0)
```

```
w0 = torch.tensor(3.0, requires_grad=True)
w1 = torch.tensor(4.0, requires_grad=True)
```

```
# Forward pass and compute the loss
```

```
h = x * w1 + w0
loss = (h - t)**2
```

```
print(loss)
```

```
tensor(36., grad_fn=<PowBackward0>)
```

```
# Backward pass
```

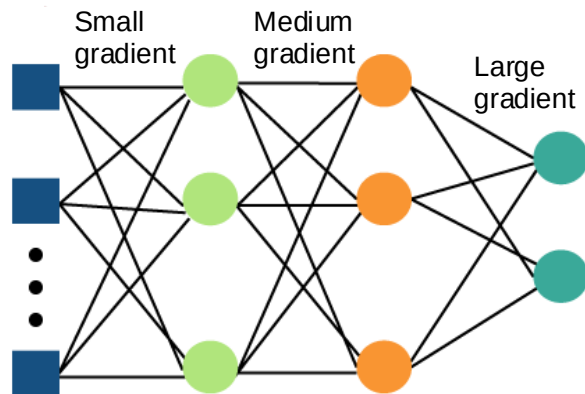
```
loss.backward()
```

```
print(w0.grad)
print(w1.grad)
```

```
tensor(12.)
tensor(24.)
```

## Gradient vanishing issue

- Deep neural networks with sigmoid or hyperbolic units often suffer from vanishing gradient.

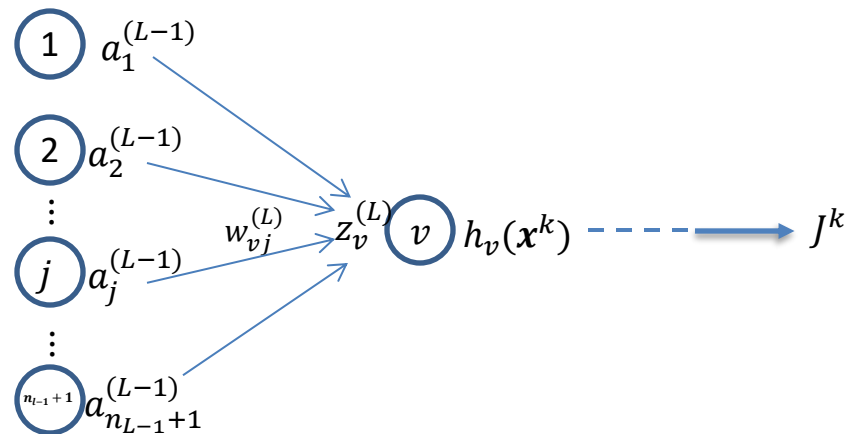
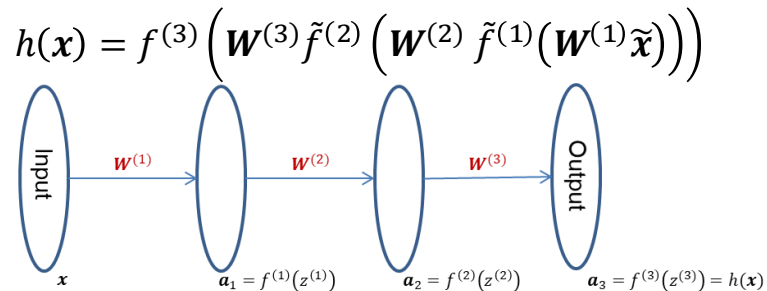


- Further back you go, the gradient tends to be smaller and smaller and smaller and smaller ...
- Roughly speaking: If you have a 'really deep neural network', the bottom layers do not get trained because the derivatives of those weights are negligible (close to zero).
- In the past, this was a significant problem that prevented researchers from effectively training these multi-layered neural networks.

## Gradient vanishing issue with $L = 3$

$$l = L = 3$$

$$\begin{aligned} \frac{\partial J^k}{\partial w_{vj}^{(3)}} &= \frac{\partial J^k}{\partial z_v^{(3)}} \frac{\partial z_v^{(3)}}{\partial w_{vj}^{(3)}} \\ &= \frac{\partial J^k}{\partial z_v^{(3)}} a_j^{(2)} \\ &= \frac{\partial J^k}{\partial h_v(x^k)} \frac{\partial h_v(x^k)}{\partial z_v^{(3)}} a_j^{(2)} \\ &= \frac{\partial J^k}{\partial h_v(x^k)} \frac{\partial f^{(3)}(z_v^{(3)})}{\partial z_v^{(3)}} a_j^{(2)} \\ &= \frac{\partial J^k}{\partial h_v(x^k)} f'^{(3)}(z_v^{(3)}) a_j^{(2)} \end{aligned}$$



## Gradient vanishing issue with $L = 3$

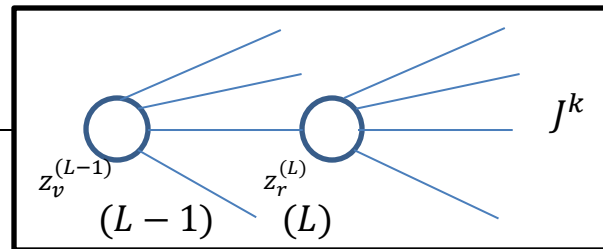
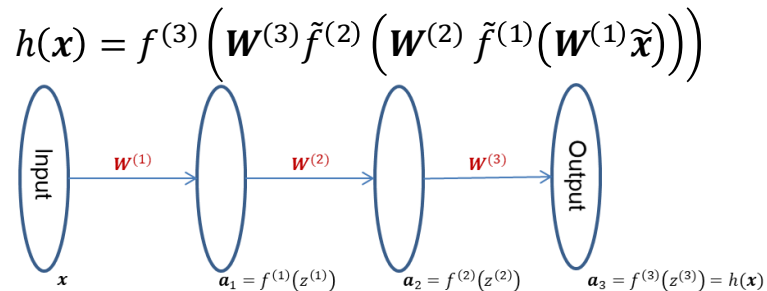
$$l = L - 1 = 2$$

$$\frac{\partial J^k}{\partial w_{vj}^{(L-1)}} = \frac{\partial J^k}{\partial z_v^{(L-1)}} a_j^{(L-2)}$$

$$= \left( \sum_{r \in (L)} \frac{\partial J^k}{\partial z_r^{(L)}} \frac{\partial z_r^{(L)}}{\partial z_v^{(L-1)}} \right) a_j^{(L-2)}$$

$$= \left( \sum_{r \in (L)} \frac{\partial J^k}{\partial z_r^{(L)}} w_{rv} f'^{(L-1)}(z_v^{(L-1)}) \right) a_j^{(L-2)}$$

$$= f'^{(L-1)}(z_v^{(L-1)}) \left( \sum_{r \in (3)} w_{rv} \frac{\partial J^k}{\partial z_r^{(L)}} \right) a_j^{(L-2)}$$



$$\begin{aligned} \frac{\partial z_r^{(L)}}{\partial z_v^{(L-1)}} &= \frac{\sum_{s \in (L-1)} w_{rs} a_s^{(L-1)}}{\partial z_v^{(L-1)}} = \frac{\sum_{s \in (L-1)} w_{rs} f^{(L-1)}(z_s^{(L-1)})}{\partial z_v^{(L-1)}} \\ &= w_{rv} f'^{(L-1)}(z_v^{(L-1)}) \end{aligned}$$

The quantity  $\frac{\partial J^k}{\partial z_r^{(L)}}$  is already known! i.e. 'back propagation of the guardian'

## Gradient vanishing issue with $L = 3$

$$l = L - 2 = 1$$

$$\frac{\partial J^k}{\partial w_{vj}^{(1)}} = \frac{\partial J^k}{\partial z_v^{(1)}} x_j$$

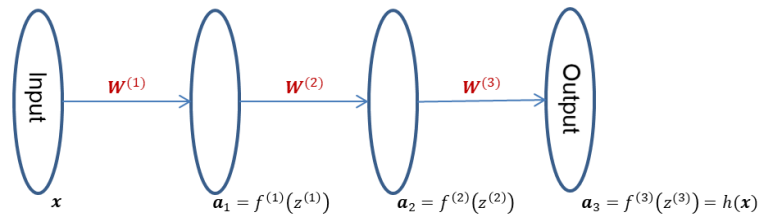
$$= \left( \sum_{r \in (2)} \frac{\partial J^k}{\partial z_r^{(2)}} \frac{\partial z_r^{(2)}}{\partial z_v^{(1)}} \right) x_j$$

$$= \left( \sum_{r \in (2)} \frac{\partial J^k}{\partial z_r^{(2)}} w_{rv} f'^{(1)}(z_v^{(1)}) \right) x_j$$

$$= f'^{(1)}(z_v^{(1)}) \left( \sum_{r \in (2)} w_{rv} \frac{\partial J^k}{\partial z_r^{(2)}} \right) x_j$$

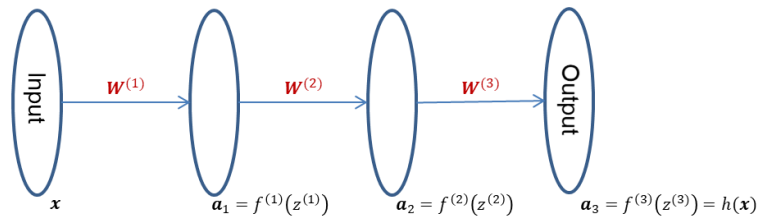
The quantity  $\frac{\partial J^k}{\partial z_r^{(2)}}$  is already known! i.e. 'back propagation of the guardian'

$$h(x) = f^{(3)} \left( \mathbf{w}^{(3)} \tilde{f}^{(2)} \left( \mathbf{w}^{(2)} \tilde{f}^{(1)} \left( \mathbf{w}^{(1)} \tilde{x} \right) \right) \right)$$



## Gradient vanishing issue with $L = 3$

$$h(\mathbf{x}) = f^{(3)}\left(\mathbf{w}^{(3)} \tilde{f}^{(2)}\left(\mathbf{w}^{(2)} \tilde{f}^{(1)}\left(\mathbf{w}^{(1)} \tilde{\mathbf{x}}\right)\right)\right)$$



$$\frac{\partial J^k}{\partial z_v^{(3)}} = -2 \left( t_v^k - h_v(\mathbf{x}^k) \right) f'^{(3)}(z_v^{(3)})$$

$$\frac{\partial J^k}{\partial z_v^{(2)}} = f'^{(2)}(z_v^{(2)}) \sum_{r \in (3)} w_{rv} \frac{\partial J^k}{\partial z_r^{(3)}}$$

$$\frac{\partial J^k}{\partial z_v^{(1)}} = f'^{(1)}(z_v^{(1)}) \sum_{r \in (2)} w_{rv} \frac{\partial J^k}{\partial z_r^{(2)}}$$



$$\frac{\partial J^k}{\partial w_{vj}^{(3)}} = \left[ -2 \left( t_v^k - h_v(\mathbf{x}^k) \right) f'^{(3)}(z_v^{(3)}) \right] a_j^{(2)}$$

$$\frac{\partial J^k}{\partial w_{vj}^{(2)}} = \left[ f'^{(2)}(z_v^{(2)}) \sum_{r \in (l+1)} w_{rv} \left( 2 \left( t_v^k - h_v(\mathbf{x}^k) \right) f'^{(3)}(z_v^{(3)}) \right) \right] a_j^{(1)}$$

$$\frac{\partial J^k}{\partial w_{vj}^{(1)}} = \left[ f'^{(1)}(z_v^{(1)}) \sum_{r \in (l+1)} w_{rv} \left( f'^{(2)}(z_v^{(2)}) \sum_{r \in (l+1)} w_{rv} \left( 2 \left( t_v^k - h_v(\mathbf{x}^k) \right) f'^{(3)}(z_v^{(3)}) \right) \right) \right] x_j$$

To compute the gradient at the  $L = 1$  layer, the derivative of the activation function is called  $L$  times. Roughly speaking we have:

$$\frac{\partial J(\mathbf{W}, \mathbf{x}, \mathbf{y})}{\partial w_{vj}^{(1)}} \propto (\dots) f'^{(L)}(z^{(L)}) \times (\dots) f'^{(L-1)}(z^{(L-1)}) \times \dots \times (\dots) f'^{(2)}(z^{(2)}) \times (\dots) f'^{(1)}(z^{(1)})$$

# Gradient vanishing issue

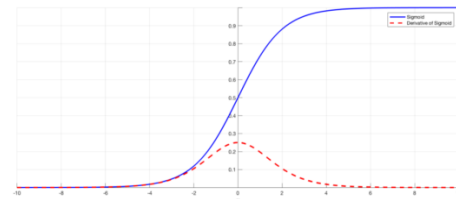
- To compute the gradient at the input layer, the derivative of the activation function is called  $L$  times. Roughly speaking we have:

$$\frac{\partial J(\mathbf{W}, \mathbf{x}, \mathbf{y})}{\partial w_{vj}^{(1)}} \propto (\dots) f'^{(L)}(z^{(L)}) \times (\dots) f'^{(L-1)}(z^{(L-1)}) \times \dots \times (\dots) f'^{(2)}(z^{(2)}) \times (\dots) f'^{(1)}(z^{(1)})$$

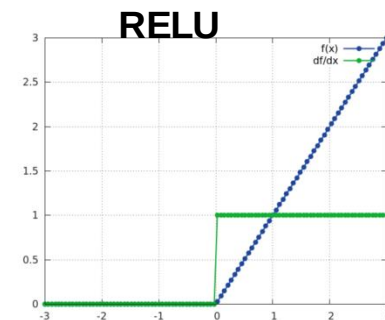
- If the  $f'^{(i)}(z_v^{(i)})$  are smaller than one then  $\frac{\partial J(\mathbf{W}, \mathbf{x}, \mathbf{y})}{\partial w_{vj}^{(1)}}$  converge exponentially rapidly to 0 when  $L$  increase.

- example if  $a=0.2$  then  $a^{10} \approx 10^{-7}$

- The maximum of the derivative of the sigmoidal function is 0.25
  - Gradient vanishing issue

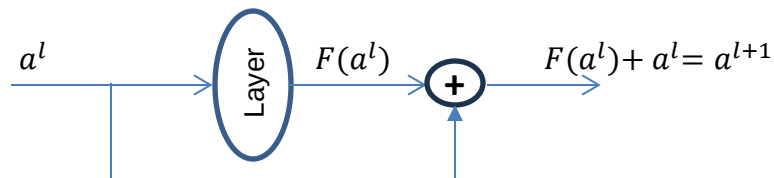


- Several popular solutions to avoid vanishing issue :
  - Pre-training
  - Rectified linear units (RELU)
  - Residual neural network (ResNet) – Skip connections
  - Batch normalization
- but the RELU derivative is 0 if input negative - 'dead neuron' issue
  - cf Demo: Tensorflow playground

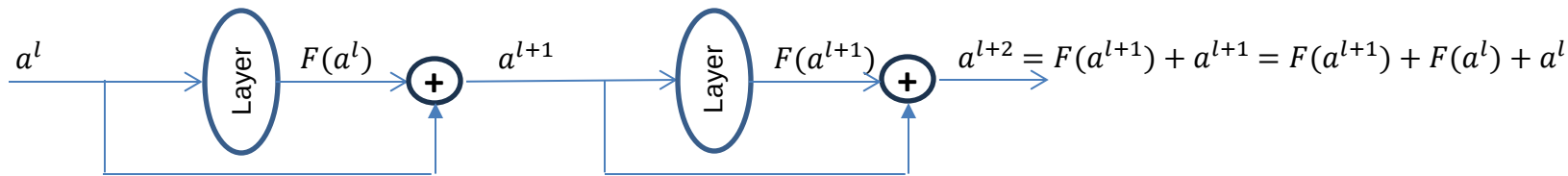




## Residual neural network (ResNet)



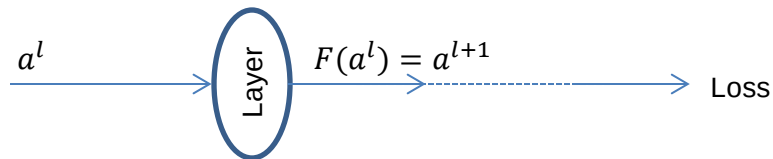
Applying this formulation recursively: (e.g.  $a^{l+2}$ )



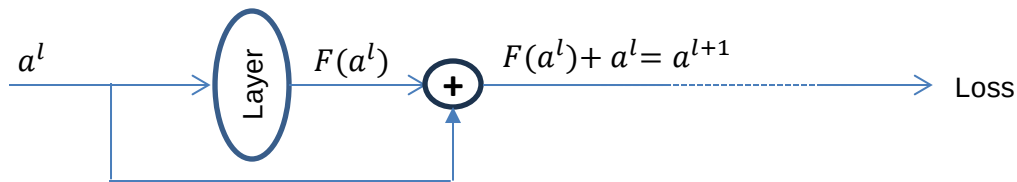
The general relationship:

$$a^L = a^l + \sum_{i=l}^{L-1} F(a^i)$$

## Residual neural network (ResNet)



$$\begin{aligned}\frac{\partial \text{Loss}}{\partial a^l} &= \frac{\partial \text{Loss}}{\partial a^{l+1}} \times \frac{\partial a^{l+1}}{\partial a^l} \\ &= \frac{\partial \text{Loss}}{\partial a^{l+1}} \times \frac{\partial F(a^l)}{\partial a^l}\end{aligned}$$



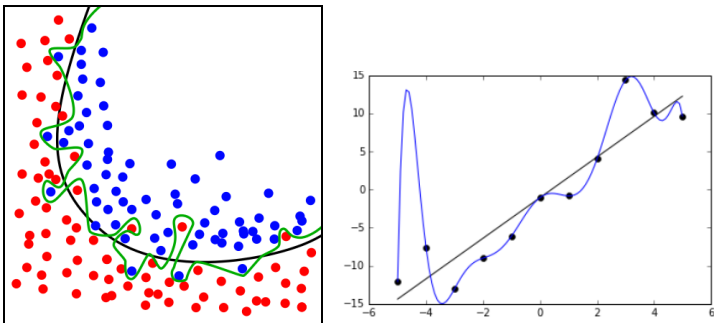
$$\begin{aligned}\frac{\partial \text{Loss}}{\partial a^l} &= \frac{\partial \text{Loss}}{\partial a^{l+1}} \times \frac{\partial a^{l+1}}{\partial a^l} \\ &= \frac{\partial \text{Loss}}{\partial a^{l+1}} \times \frac{\partial (F(a^l) + a^l)}{\partial a^l} \\ &= \frac{\partial \text{Loss}}{\partial a^{l+1}} \times \left( \frac{\partial F(a^l)}{\partial a^l} + 1 \right) \\ &= \frac{\partial \text{Loss}}{\partial a^{l+1}} \times \frac{\partial F(a^l)}{\partial a^l} + \frac{\partial \text{Loss}}{\partial a^{l+1}}\end{aligned}$$

This formulation suggests that: Even if the gradients of the  $F(a^l)$  terms are small, the total gradient  $\frac{\partial \text{Loss}}{\partial a^l}$  resists vanishing thanks to the added term  $\frac{\partial \text{Loss}}{\partial a^{l+1}}$ .

# Overfitting

**Overfitting** is "the production of an analysis that corresponds too closely or exactly to a particular set of data, and may therefore fail to fit additional data or predict future observations reliably"

(Definition of "overfitting" at OxfordDictionaries.com)



While the **green** (classification) & **blue** (regression) lines best follow the training data, it is too dependent on that data and it is likely to have a higher error rate on new unseen data, compared to the black line.

- Roughly speaking, overfitting occurs when the model class (i.e., the architecture) is too complex for the learning problem.
  - the algorithm starts to learn the noise
- Overfitting can be measured during learning via a validation set (not used during gradient descent).
- To avoid it:
  - Regularization, mainly L1 and L2
  - Grid-search, to search optimal architecture
    - difficult in deep learning
  - Early stopping
  - Dropout nodes in the deep learning architecture
  - ...

# How to deal with overfitting ?

Options:

- Reducing the number of parameters ( $\sim$ complexity)
  - Model selection (Validation set, Cross validation, ...)
    - Traditional method but difficult with DL
- Keep all parameters with ...
  - Early stopping
  - Dropout
  - Regularization
  - ...

# Overfitting – validation set

In [2]: (x\_train, y\_train), (x\_test, y\_test) = keras.datasets.mnist.load\_data()

```
# Scale images variables in [0, 1]
x_train = x_train.astype("float32") / 255
x_test = x_test.astype("float32") / 255

# shape (60000, 28, 28) -> (60000, 28, 28, 1)
x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)

# shape (60000,) -> (60000, 1)
y_train = y_train.reshape(-1,1)
y_test = y_test.reshape(-1,1)

# output to one hot encoding
enc = OneHotEncoder(sparse=False)
y_train = enc.fit_transform(y_train)
y_test = enc.transform(y_test)
```

```
In [3]: model = Sequential([
    Flatten(input_shape=(28,28,1)),
    Dense(30, activation='relu'),
    Dense(20, activation='relu'),
    Dense(15, activation='relu'),
    Dense(10, activation='softmax')
])
model.summary()
model.compile(optimizer='adam', loss='categorical_crossentropy')
hist = model.fit(x_train, y_train, epochs=30, verbose=1, validation_split=0.2)
```

Model: "sequential"

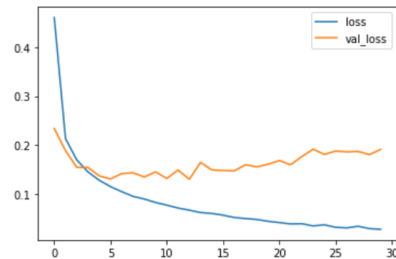
| Layer (type)             | Output Shape | Param # |
|--------------------------|--------------|---------|
| flatten (Flatten)        | (None, 784)  | 0       |
| dense_1 (Dense)          | (None, 30)   | 23550   |
| dense_2 (Dense)          | (None, 20)   | 620     |
| dense_3 (Dense)          | (None, 15)   | 315     |
| dense_4 (Dense)          | (None, 10)   | 160     |
| Total params: 24,645     |              |         |
| Trainable params: 24,645 |              |         |
| Non-trainable params: 0  |              |         |

## MNIST



In [4]: pd.DataFrame(hist.history).plot()

Out[4]: <AxesSubplot:>



In [5]: model.evaluate(x\_test, y\_test)

313/313 [=====] - 1s 5ms/step - loss: 0.1920

Out[5]: 0.1919853687286377

## Overfitting – Early stopping (i)

```
In [3]: early_stopping = EarlyStopping(monitor='val_loss',
                                       patience=3,
                                       verbose=1)
checkpoint = ModelCheckpoint(filepath = 'checkpoint_best',
                             monitor='val_loss',
                             save_best_only=True,
                             save_weights_only=True,
                             verbose=1)
```

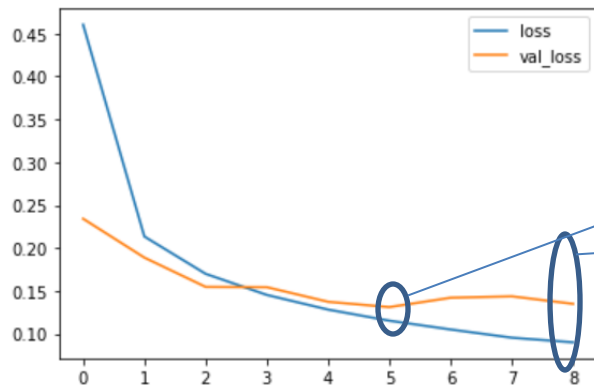
```
In [4]: model = Sequential([
        Flatten(input_shape=(28,28,1)),
        Dense(30, activation='relu'),
        Dense(20, activation='relu'),
        Dense(15, activation='relu'),
        Dense(10, activation='softmax')
    ])
model.summary()
model.compile(optimizer='adam', loss='categorical_crossentropy')
hist = model.fit(x_train, y_train, epochs=30, verbose=1, validation_split=0.2,
                callbacks=[early_stopping, checkpoint])
```

```
Epoch 1/30
1499/1500 [=====>.] - ETA: 0s - loss: 0.4605
Epoch 00001: val_loss improved from inf to 0.23411, saving model to checkpoint_best
1500/1500 [=====] - 17s 11ms/step - loss: 0.4604 - val_loss: 0.2341
Epoch 2/30
1499/1500 [=====>.] - ETA: 0s - loss: 0.2133
Epoch 00002: val_loss improved from 0.23411 to 0.18881, saving model to checkpoint_best
1500/1500 [=====] - 17s 11ms/step - loss: 0.2134 - val_loss: 0.1888
Epoch 3/30
1497/1500 [=====>.] - ETA: 0s - loss: 0.1701
Epoch 00003: val_loss improved from 0.18881 to 0.15467, saving model to checkpoint_best
1500/1500 [=====] - 16s 11ms/step - loss: 0.1699 - val_loss: 0.1547
Epoch 4/30
1497/1500 [=====>.] - ETA: 0s - loss: 0.1449
Epoch 00004: val_loss improved from 0.15467 to 0.15432, saving model to checkpoint_best
1500/1500 [=====] - 19s 13ms/step - loss: 0.1453 - val_loss: 0.1543
Epoch 5/30
1499/1500 [=====>.] - ETA: 0s - loss: 0.1281
Epoch 00005: val_loss improved from 0.15432 to 0.13729, saving model to checkpoint_best
1500/1500 [=====] - 19s 12ms/step - loss: 0.1282 - val_loss: 0.1373
Epoch 6/30
1500/1500 [=====] - ETA: 0s - loss: 0.1153
Epoch 00006: val_loss improved from 0.13729 to 0.13111, saving model to checkpoint_best
1500/1500 [=====] - 21s 14ms/step - loss: 0.1153 - val_loss: 0.1311
Epoch 7/30
1493/1500 [=====>.] - ETA: 0s - loss: 0.1048
Epoch 00007: val_loss did not improve from 0.13111
1500/1500 [=====] - 15s 10ms/step - loss: 0.1050 - val_loss: 0.1420
Epoch 8/30
1496/1500 [=====>.] - ETA: 0s - loss: 0.0954
Epoch 00008: val_loss did not improve from 0.13111
1500/1500 [=====] - 18s 12ms/step - loss: 0.0954 - val_loss: 0.1437
Epoch 9/30
1497/1500 [=====>.] - ETA: 0s - loss: 0.0899
Epoch 00009: val_loss did not improve from 0.13111
1500/1500 [=====] - 17s 11ms/step - loss: 0.0900 - val_loss: 0.1349
Epoch 00009: early stopping
```

## Overfitting – Early stopping (ii)

```
In [5]: pd.DataFrame(hist.history).plot()
```

```
Out[5]: <AxesSubplot:>
```



Save the model minimizing val\_loss

```
checkpoint = ModelCheckpoint(filepath = 'checkpoint_best',  
                             monitor='val_loss',  
                             save_best_only=True,  
                             save_weights_only=True,  
                             verbose=1)
```

Stop the learning process after 3 consecutive increases in val\_loss

```
early_stopping = EarlyStopping(monitor='val_loss',  
                               patience=3,  
                               verbose=1)
```

```
In [6]: model.load_weights('checkpoint_best')  
        model.evaluate(x_test, y_test)
```

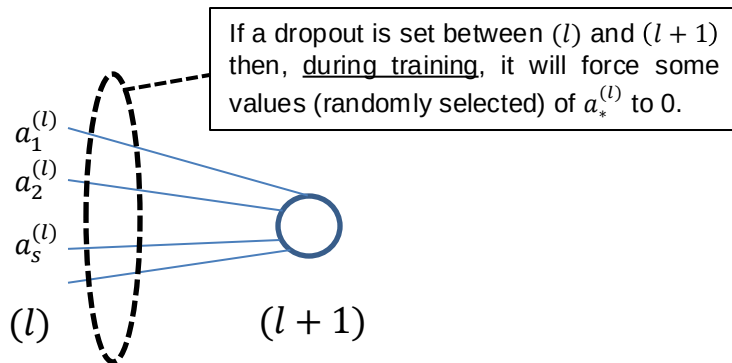
```
313/313 [=====] - 2s 6ms/step - loss: 0.1296
```

```
Out[6]: 0.12957791984081268
```

Reload the best model minimizing val\_loss

## Overfitting – Dropout (i)

- At each step during the training time, the **dropout layer** randomly sets the input nodes (" $a$ ") to 0 with a given frequency.



- The input nodes not set to 0 are scaled up by  $1/(1 - \text{rate})$  such that the sum over all inputs is unchanged.
- The Dropout layer is inactive during predictions.
- It helps to generalize by preventing a node from becoming too important in the prediction.
- It can also be interpreted as a way to learn in parallel a set of neural networks with different architectures.
  - a kind of bagging technique like random forest



# Overfitting – Dropout (ii)

[https://github.com/ocaelen/intro\\_deeplearning/blob/main/tf\\_simple\\_dropout.ipynb](https://github.com/ocaelen/intro_deeplearning/blob/main/tf_simple_dropout.ipynb)

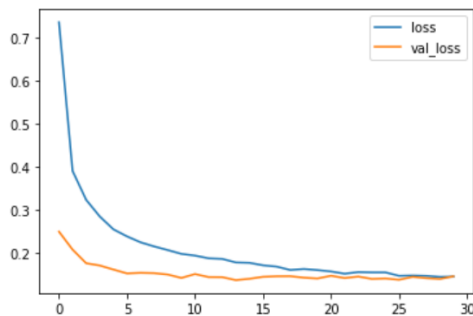
```
In [3]: model = Sequential([
        Flatten(input_shape=(28,28,1)),
        Dense(30, activation='relu'),
        Dropout(0.1),
        Dense(20, activation='relu'),
        Dropout(0.1),
        Dense(15, activation='relu'),
        Dropout(0.1),
        Dense(10, activation='softmax')
    ])
model.summary()
model.compile(optimizer='adam', loss='categorical_crossentropy')
hist = model.fit(x_train, y_train, epochs=30, verbose=1, validation_split=0.2)
```

Model: "sequential"

| Layer (type)             | Output Shape | Param # |
|--------------------------|--------------|---------|
| =====                    |              |         |
| flatten (Flatten)        | (None, 784)  | 0       |
| =====                    |              |         |
| dense (Dense)            | (None, 30)   | 23550   |
| =====                    |              |         |
| dropout (Dropout)        | (None, 30)   | 0       |
| =====                    |              |         |
| dense_1 (Dense)          | (None, 20)   | 620     |
| =====                    |              |         |
| dropout_1 (Dropout)      | (None, 20)   | 0       |
| =====                    |              |         |
| dense_2 (Dense)          | (None, 15)   | 315     |
| =====                    |              |         |
| dropout_2 (Dropout)      | (None, 15)   | 0       |
| =====                    |              |         |
| dense_3 (Dense)          | (None, 10)   | 160     |
| =====                    |              |         |
| Total params: 24,645     |              |         |
| Trainable params: 24,645 |              |         |
| Non-trainable params: 0  |              |         |

```
In [4]: pd.DataFrame(hist.history).plot()
```

Out[4]: <AxesSubplot:>



```
In [5]: model.evaluate(x_test, y_test)
```

313/313 [=====] - 1s 3ms/step - loss: 0.1505

Out[5]: 0.15052251517772675

# Regularization

We keep all parameters but reduce the magnitude of the parameters  $w_{vj}^{(l)}$ .

A regularization term  $R(h)$  is added to the loss function

$$J_R(\mathbf{w}, D_N) = \left( \frac{1}{N} \sum_{i=1}^N J^i \right) + \lambda R(h)$$

where  $J^i$  is the underlying loss that describes the cost of predicting  $h(\mathbf{x}^i)$  when the true output is  $t^i$  and  $\lambda$  is a parameter which controls the importance of the regularization term  $R(h)$ .

Given the training set  $D_N$ , during learning we search the optimal parameters  $\mathbf{w}$  minimizing this new regularized loss function.

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} \left( \frac{1}{N} \sum_{i=1}^N J^i \right) + \lambda R(h)$$

## L1 & L2 Regularization

The key difference between L1 & L2 is the penalty term.

$$J_{l1}(\mathbf{w}, D_N) = \left( \frac{1}{N} \sum_{i=1}^N J^i \right) + \lambda \sum_j \|w_j\|$$

$$J_{l2}(\mathbf{w}, D_N) = \left( \frac{1}{N} \sum_{i=1}^N J^i \right) + \lambda \sum_j (w_j)^2$$

- If  $\lambda = 0$  then we get back the original loss function (overfitting if lot of parameters).
- However, if  $\lambda$  is very large then it will add too much weight and it will lead to underfitting.

**Back to Single-layer Neural Networks** (i.e., Linear regression & logistic regression)

- If L1 regularization, then “Lasso Regression”
- If L2 regularization, then “Ridge Regression”

# It is essential to normalize or standardize the input variables!

$$\mathbf{X} = \begin{pmatrix} x_1^1 & \cdots & x_n^1 \\ \vdots & \ddots & \vdots \\ x_1^N & \cdots & x_n^N \end{pmatrix} = (\mathbf{x}_1 \quad \dots \quad \mathbf{x}_n) \in \mathbb{R}^{N \times n}$$

- Each column of the training dataset can have different quantities with very different ranges of values.

- size in meters : 1.5  $\rightarrow$  2.2
- weight in kg: 50  $\rightarrow$  180
- incomes in €: 700  $\rightarrow$  4000

- the objective is to bring all variables to the same scale.

- Normalize:

$$\tilde{x}_j^i = \frac{x_j^i - \min(\mathbf{x}_j)}{\max(\mathbf{x}_j) - \min(\mathbf{x}_j)}$$

- Standardize:

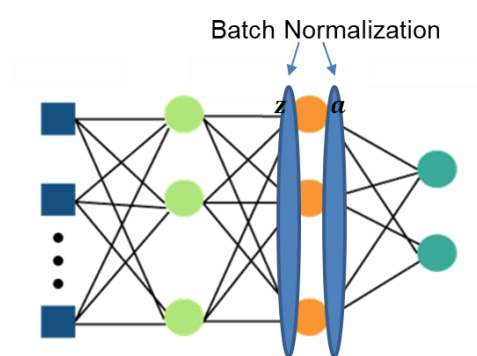
$$\tilde{x}_j^i = \frac{x_j^i - \text{avg}(\mathbf{x}_j)}{\text{sd}(\mathbf{x}_j)}$$

- It helps improve model performance, training stability, and convergence speed.
- By scaling inputs to a consistent range gradient descent optimization becomes more efficient, preventing large weights.
- This uniformity in scale ensures that each feature contributes equally to the learning process, reducing the risk of certain features dominating others.

# Batch Normalization

*“Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”*  
(2015)

- The learning performance is improved when the input variables are normalized.
  - Why not apply the same normalization operation to the hidden layers?
- The purpose of this idea is to normalize either the  $a = f(\mathbf{w}^T \tilde{\mathbf{x}})$  or the  $z = \mathbf{w}^T \tilde{\mathbf{x}}$ .
  - In the original paper, it is on the  $z$ .
- Ideally, the normalization would be conducted over the entire training set, but to use this step jointly with stochastic optimization methods, it is impractical to use the global information.



# Batch Normalization

Let's assume that:

- we normalize the  $z$
- $|\mathcal{B}|$  is the batch size

The Batch Normalization is defined as follow:

$$z_{norm} = \frac{z - \hat{\mu}_z}{\sqrt{\hat{\sigma}^2 + \epsilon}}; \quad \text{where } \hat{\mu}_z = \frac{1}{|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} z_i \quad \text{and} \quad \hat{\sigma}^2 = \frac{1}{|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} (z_i - \hat{\mu}_z)^2$$

and

$$\tilde{z}_{norm} = \gamma \cdot z_{norm} + \beta$$

This is done for each neuron in the layer.

The  $\epsilon$  is added in the denominator for numerical stability and is an arbitrarily small constant.

$\gamma$  and  $\beta$  are learnable parameters.

Note that because we subtract the mean  $\hat{\mu}_z$ , we don't really need extra bias parameter  $w_0$  in the neurone because it is loss.

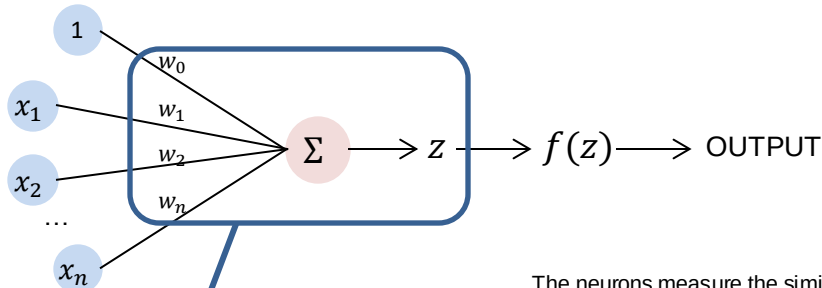
→  $z - \hat{\mu}_z$  doesn't depend on the bias, and so  $z_{norm}$  doesn't depend on the bias. The bias becomes irrelevant.

→  $\beta$  takes the role of the bias

# Batch Normalization – during training & inference

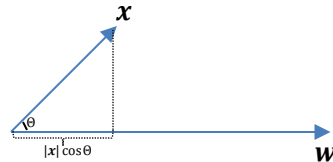
- **During training:**
  - Batch normalization helps to stabilize and speed up the learning process by normalizing the inputs of layers.
  - After normalization, a learnable scale  $\gamma$  and shift parameter  $\beta$  is applied to maintain the representation power of the model.
  - The mean and variance of each batch are also used to compute a running average, which is an estimate of the overall mean and variance of the dataset.
- **During inference (or the test phase):**
  - The batch normalization layer behaves differently.
  - Since we usually evaluate or predict one sample at a time, it's not appropriate to compute the mean and variance from the current 'test batch'.
  - Instead, we use the running average of mean and variance calculated during the training phase to normalize the inputs.
  - This running average serves as an estimate of the population means and variance and ensures that the network's behaviour remains consistent between training and inference.

## The inner product as similarity measure



Inner product  
 $z = \mathbf{w}^T \tilde{\mathbf{x}}^i$

$$\mathbf{w}^T \tilde{\mathbf{x}}^i = |\mathbf{w}| |\mathbf{x}| \cos(\theta)$$



The neurons measure the similarity between the neuron's input vector  $\mathbf{x}$  and the synaptic weights  $\mathbf{w}$ .

- Neurons search in input for patterns defined by synaptic weights.
- the output of the neuron will be large if the input is like the pattern defined by the weights.

**Inner products** are often used as a **measure of similarity between two vectors** because they can provide a concise and computationally efficient way to **compare the directions of the vectors**.

When two vectors are combined through an inner product, the result is a scalar value that reflects the extent to which the vectors are pointing in the same direction.

**If the two vectors are pointing in the same direction**, the angle between them will be 0 degrees, and the  $\cos(0)=1$ . This means that the inner product of the two vectors will be equal to the product of their magnitudes, **which is at its maximum value**.

On the other hand, **if the two vectors are orthogonal** (i.e., perpendicular) to each other, the angle between them is 90 degrees, and the cosine of 90 is 0. This means that **the inner product of the two vectors will be 0**, which indicates that they are not similar at all.

**In general, the larger the inner product of two vectors, the more similar they are in terms of direction.**